University of Trieste

Data Science and Artificial Intelligence

Mathematics, Informatics and Geosciences

Reinforcement Learning

Course Notes

Author:
Ester De Giosa
Alina Fogar
Riccardo Samaritan

migesolidale@gmail.com

Lecturer:
Prof. Antonio Celani

October 19, 2026

Contents

1 Introduction **3**
- 1.1 Reinforcement Learning . . . 3
  - 1.1.1 Elements of reinforcement . . . 3
  - 1.1.2 Temporal Difference Learning . . . 4
  - 1.1.3 Multi-Armed Bandit Problem . . . 4

2 Markov Decision Processes **8**
- 2.1 Interface . . . 8
- 2.2 Markov Decision Process . . . 9
- 2.3 Fully Observable Setting . . . 9
- 2.4 Partially Observable Markov Decision Process (POMDP) . . . 10
  - 2.4.1 Observation Model . . . 11
  - 2.4.2 Perfect Memory and Information State . . . 11
  - 2.4.3 Belief States and the Belief-MDP . . . 12
  - 2.4.4 Bellman Equation in Belief Space . . . 13
- 2.5 Goals and Rewards . . . 13
- 2.6 Returns and Episodes . . . 13
- 2.7 Unified Notation for Episodic and Continuing Tasks . . . 14
- 2.8 Policies and Value Functions . . . 14
- 2.9 Optimal Policies and Value Functions . . . 15

3 Dynamic Programming **17**
- 3.1 Policy Evaluation . . . 17
- 3.2 Policy Improvement . . . 18
- 3.3 Policy Iteration . . . 19
- 3.4 Comparison with Value Iteration . . . 20
- 3.5 Value Iteration . . . 21
- 3.6 Asynchronous DP . . . 22
- 3.7 Generalized Policy Iteration . . . 22
- 3.8 Linear Programming for MDP . . . 22

4 Monte Carlo Methods **25**
- 4.1 Monte Carlo Prediction . . . 25
- 4.2 Monte Carlo Estimation of Action Values . . . 26
- 4.3 Monte Carlo Control . . . 26
- 4.4 Monte Carlo Control without Exploring Starts . . . 27
- 4.5 Off-policy Prediction via Importance Sampling . . . 28
- 4.6 Incremental Implementation . . . 29
- 4.7 Off-policy Monte Carlo Control . . . 29

5 Temporal-Difference Learning **31**
- 5.1 TD Prediction . . . 31
- 5.2 Advantages of TD Prediction Methods . . . 32
- 5.3 Optimality of TD(0) . . . 32
- 5.4 Sarsa: On-policy TD Control . . . 32
- 5.5 Q-learning: Off-policy TD Control . . . 33
- 5.6 Expected Sarsa . . . 34
- 5.7 Maximization Bias and Double Learning . . . 35

6 n-step Bootstrapping **37**

6.1 n-step TD Prediction . . . 37
6.2 n-step Sarsa . . . 39
6.3 n-step Off-policy Learning . . . 39

7 Planning and Learning with Tabular Methods **41**
7.1 Models and Planning . . . 41
7.2 Dyna: Integrated Planning, Acting, and Learning . . . 42
7.3 When the Model Is Wrong . . . 43
7.4 Prioritized Sweeping . . . 44
7.5 Expected vs. Sample Updates . . . 45
7.6 Trajectory Sampling . . . 46
7.7 Real-time Dynamic Programming . . . 46
7.8 Planning at Decision Time . . . 47
7.9 Heuristic Search . . . 47
7.10 Rollout Algorithms . . . 48
7.11 Monte Carlo Tree Search . . . 48

8 Approximate Solution Methods **50**
8.1 Value-function Approximation . . . 50
8.2 The Prediction Objective ($\overline{VE}$) . . . 50
8.3 Stochastic-gradient and Semi-gradient Methods . . . 51
8.3.1 Stochastic Approximation and Mean-Square Convergence . . . 52
8.4 Linear Methods . . . 53
8.5 Feature Construction for Linear Methods . . . 55

9 Policy Gradient Methods **56**
9.1 Policy Approximation and its Advantages . . . 56
9.2 Policy Gradient Theorem . . . 56
9.3 REINFORCE: Monte Carlo Policy Gradient . . . 57
9.3.1 REINFORCE with Baseline . . . 58
9.3.2 Advantage Function . . . 59
9.3.3 Actor-Critic Methods . . . 59

10 Advanced Policy Gradient Notes **60**
10.1 Derivation of the Policy Gradient Theorem . . . 60
10.1.1 Setup and Parameterization . . . 60
10.1.2 Matrix Calculus Identity . . . 60
10.1.3 Derivation of the Policy Gradient Theorem . . . 61
10.1.4 The Log-Derivative Trick . . . 61
10.2 Global Optima in Markov Decision Processes . . . 61
10.2.1 Background and Notation . . . 61
10.2.2 The Performance Difference Lemma . . . 62
10.2.3 Local and Global Optima . . . 63

11 Multi-Agent Reinforcement Learning **64**
11.1 General Introduction . . . 64
11.1.1 Formulation . . . 64
11.1.2 Common Approaches . . . 64
11.1.3 Relation with Game Theory . . . 65

Chapter 1

Introduction

The fundamental building blocks of any decision-making system are the **agent** and the **environment**. The agent perceives the world through **sensors**, which capture both the external state of the environment and the internal state of the agent itself, and acts upon it through **actuators** - the components that translate intentions into consequences.

The agent's internal state at time t is denoted m_t, often referred to as *memory*. Its size may vary and can be decomposed into sub-variables. The environment's state at time t is denoted S_t. Together, these variables fully describe the system at any given moment.

Time is treated as **discrete** ($t \to t+1$), although working with continuous time remains possible. At each step, the agent takes an action a_t — an intention that must confront reality. Two consequences follow:

- the state of the environment (and possibly the agent) changes;
- an observation y_t is produced, which is used to update the internal state: $m_t \to m_{t+1}$.

This process then repeats. The agent's behavior is governed by a **model** of the environment — a representation we must choose carefully, as it encodes what the agent knows or assumes about the world.

1.1 Reinforcement Learning

Reinforcement Learning (RL) is the problem of learning what to do in order to maximize a numerical reward signal. The agent interracts with the enviroment without being told which actions to take; instead, it must discover which actions yield the most reward by trying them out. This distinguishes RL from supervised learning, where the agent learns from labeled input-output pairs, and from unsupervised learning, where the goal is to find patterns in data without explicit feedback. RL is therefore considered a third machine learning paradigm.

A central challenge is the trade-off between **exploration** and **exploitation**: the agent must explore the environment to discover rewarding actions, while simultaneously exploiting its current knowledge to maximize reward. All RL agents have explicit goals, can sense aspects of their environment, and can choose actions to influence it — making RL an approach that starts with a complete, interactive, goal-seeking agent.

1.1.1 Elements of reinforcement

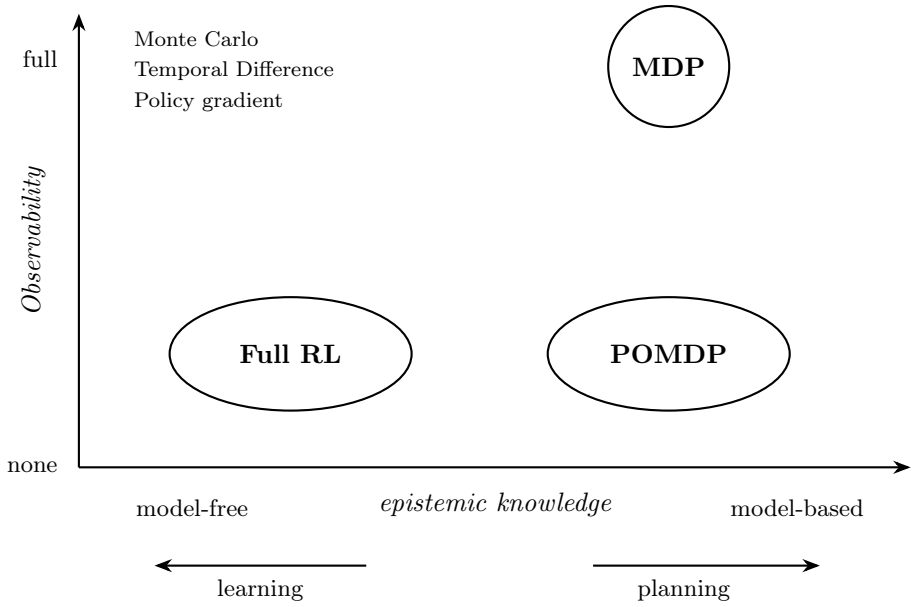
- **Policy** $\pi(a|m)$: A strategy that maps perceived states of the environment to actions. In simple cases it may be a lookup table; in others it may involve extensive computation. Policies may be stochastic, specifying a probability distribution over actions.
- **Reward signal** R_{t+1}: A scalar feedback signal received from the environment after each action. It evaluates the effectiveness of the agent's behavior and is the primary basis for updating the policy.
- **Value function**: A function that estimates the expected cumulative reward obtainable from a given state (or state-action pair). Unlike the reward signal - which reflects immediate gain — the value function captures long-term consequences, guiding the agent toward decisions that maximize expected return.
- **Model of the environment** $p(S', y \mid S, a)$: An internal representation of the environment used to predict the consequences of actions. It enables planning by simulating future states, though it is not always required.
- **Memory update** $g(m' \mid m, a, y)$: The computation that updates the agent's internal state given the previous memory, the action taken, and the observation received.

The goal is to find the optimal policy π^* that maximizes the expected cumulative reward:

$$\max_{\pi} \mathbb{E}\left[\sum_{t=0}^{T} R_{t+1}\right]$$

Most of the time the agent acts *greedily*, selecting the action that leads to the state with the highest estimated value — i.e., the highest estimated probability of winning. Occasionally, however, it selects randomly from among other actions. These are called *exploratory* moves, as they expose the agent to states it might otherwise never visit.

Taxonomy of Reinforcement Learning & Planning



1.1.2 Temporal Difference Learning

Let S_t denote the state before the greedy move, and S_{t+1} the state after. The update to the estimated value of S_t, denoted $V(S_t)$, is:

$$V(S_t) \leftarrow V(S_t) + \alpha\left(V(S_{t+1}) - V(S_t)\right)$$

where α is a small positive *learning rate*. This is known as the **temporal difference** (TD) learning rule: it updates value estimates based on the difference between the predicted value of the next state and the current estimate, without waiting for the final outcome.

1.1.3 Multi-Armed Bandit Problem

The most important feature distinguishing RL from other types of learning is that it uses training information that *evaluates* the actions taken rather than *instructs* by giving correct actions. This is what creates the need for active exploration, for an explicit search for good behavior.

Let's say we have a slot machine with multiple arms (k) and we are forced to repeatedly choose one arm to pull. After each choice we receive a numerical reward chosen from a stationary (*strong assumption!*) probability distribution that depends on the action we selected. Our objective is to maximize the expected total reward over some time period, i.e. *time steps*.

In this k-armed bandit problem, each of the k actions has an expected or mean reward given that that action is selected; this is the *value* of that action. We denote the action selected on time step t as A_t and the corresponding reward as R_t. The value then of an arbitrary action a, denoted $q_*(a)$, is the expected reward given that a is selected:

$$q_*(a) \equiv \mathbb{E}[R_t | A_t = a]$$

We assume that we don't know the action values with certainty, although we may have estimates. We denote the estimated value of action a at time step t as $Q_t(a)$. We would like $Q_t(a)$ to be close to $q_*(a)$, but we do not know the true values.

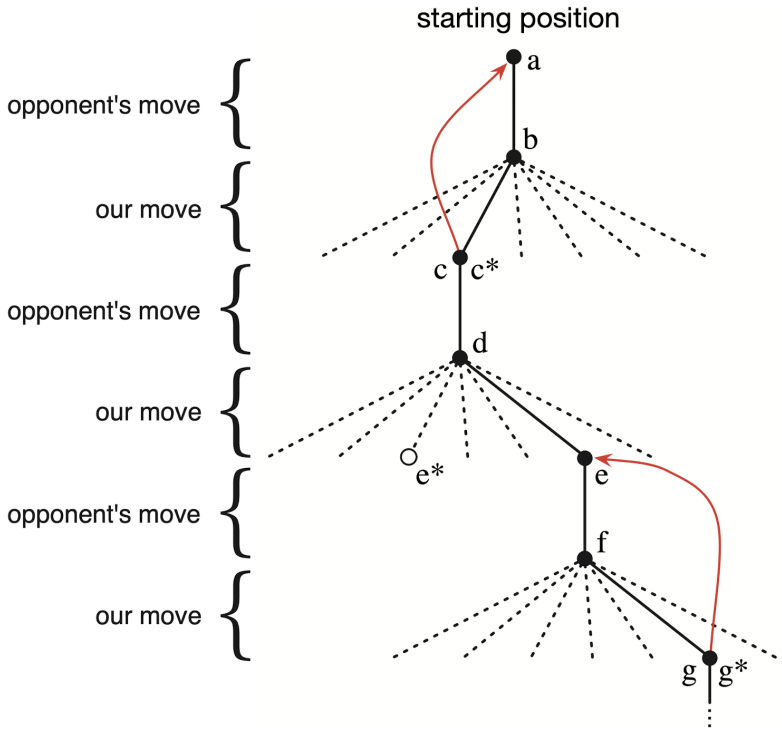


Figure 1.1: Tic Tac Toe example of a reinforcement learning agent. The agent learns to play the game by exploring different moves and receiving rewards based on the outcome of the game.

It we maintain estimates of the action values, then at any time step there is at least one action whose estimated value is greatest. We call these the *greedy* actions. When we select one of these actions, we say that we are *exploiting* our current knowledge of the values of the actions. If instead we select one of the nongreedy actions, then we say we are *exploring*, because this enables us to improve our estimate of the nongreedy action's value. Exploitation is the right thing to do to maximize the expected reward on the one step, but exploration may produce the greater total reward in the long run.

However, most of these methods make strong assumptions about stationarity and prior knowledge that are either violated or impossible to verify in most applications and in the full reinforcement learning problem that we consider in subsequent chapters. The guarantees of optimality or bounded loss for these methods are of little comfort when the assumptions of their theory do not apply.

Action-value Methods

Action-value methods are a class of reinforcement learning algorithms that focus on estimating the value of actions based on the rewards received from the environment. The key idea is to maintain an estimate of the expected reward for each action and to use this information to make decisions about which action to take next.

$$Q_t(a) \equiv \frac{\sum_{i=1}^{t-1} R_i \cdot \mathbb{1}(A_i = a)}{\sum_{i=1}^{t-1} \mathbb{1}(A_i = a)}$$

We call this the *sample-average* method for estimating action values becuase each estimate is an average of the sample of relevant rewards. To select an action, the simplest approach is to select one of the actions with the highest estimated value, i.e., one of the greedy actions as defined in the previous section. We write this *greedy* action selection method as

$$A_t \equiv \arg\max_a Q_t(a)$$

A simple alternative is to behave greedily most of the time, but every once in a while, with a small probability ϵ, instead select randomly from all the actions with equal probability, independently of the action-value estimates. These methods are named ϵ-*greedy*.

Nonstationary Bandit Problems

Previous methods assume that the action values are stationary, meaning that they do not change over time. However, in many real-world scenarios, the action values can change due to various factors such as changes in the environment or the agent's own actions. This leads to nonstationary bandit problems, where the action values

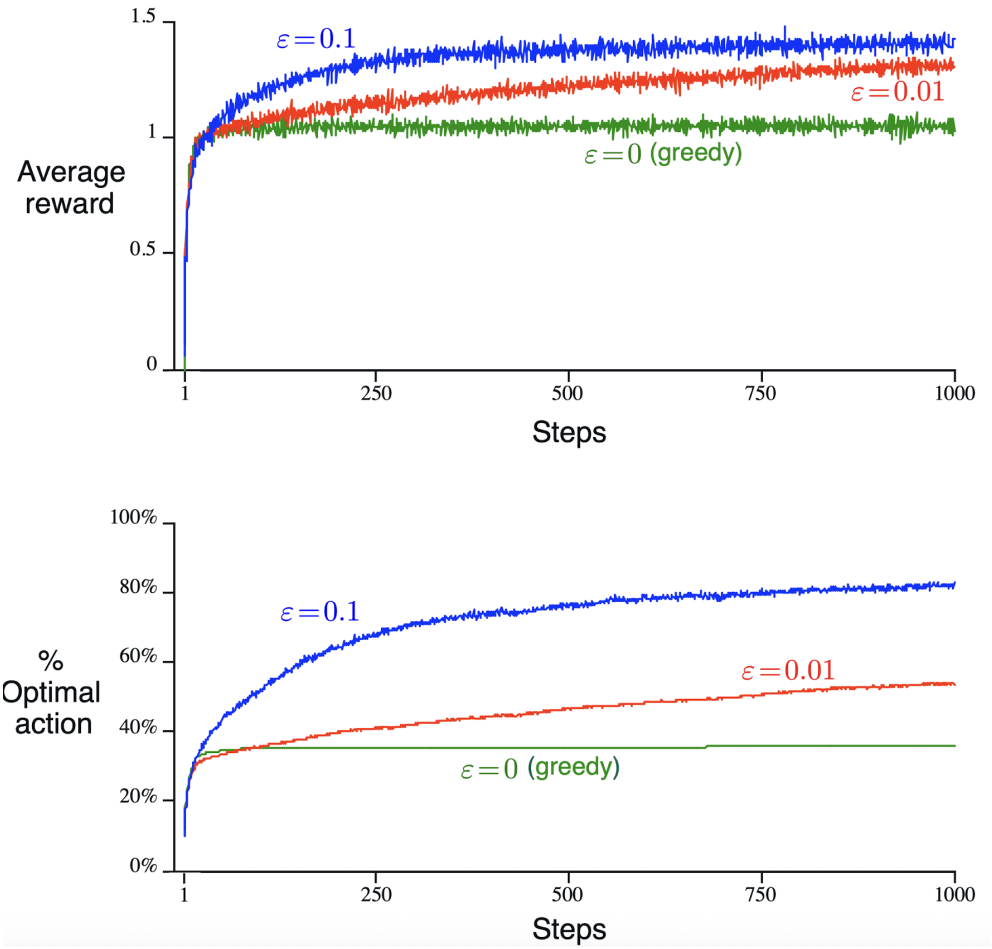


Figure 1.2: Comparison between greedy and ϵ-greedy action selection methods. The ϵ-greedy method explores the environment by selecting random actions with a small probability ϵ, while the greedy method always selects the action with the highest estimated value.

are not fixed and can vary over time. In such cases it makes sense to give more weight to recent rewards than to long-past rewards. One of the most popular ways of doing this is to use a constant step-size parameter.

$$Q_{n+1}(a) = Q_n(a) + \alpha(R_{n+1} - Q_n(a))$$

where $\alpha \in [0, 1]$ is a constant step-size parameter that determines how much weight to give to the most recent reward. This approach allows the agent to adapt to changes in the environment and to update its action-value estimates accordingly.

Optimistic Initial Values

All the methods we have discussed so far are dependent to some extent on the initial action-value estimates, $Q_1(a)$. In the language of statistics, these methods are *biased* by their initial estimates.

Initial action-values can also be used as a simple way to encourage exploration. Setting the initial action values to be optimistic, i.e., higher than the true expected rewards, can encourage the agent to explore more. This is because the agent will initially prefer actions with higher estimated values, leading to more exploration of those actions until their true values are learned. We regard it as a simple trick that can be quite effective on stationary problems, but it is far from being a generally useful approach to encouraging exploration.

Gradient Bandit Algorithms

Here we consider learning a numerical *preference* for each action a, which we denote as $H_t(a) \in \mathbb{R}$. The larger the preference, the more often that action is taken, but the preference has no interpretation in terms of reward. Only the relative preference of one action over another is important.

$$Pr\{A_t = a\} = \frac{e^{H_t(a)}}{\sum_b e^{H_t(b)}} = \pi_t(a)$$

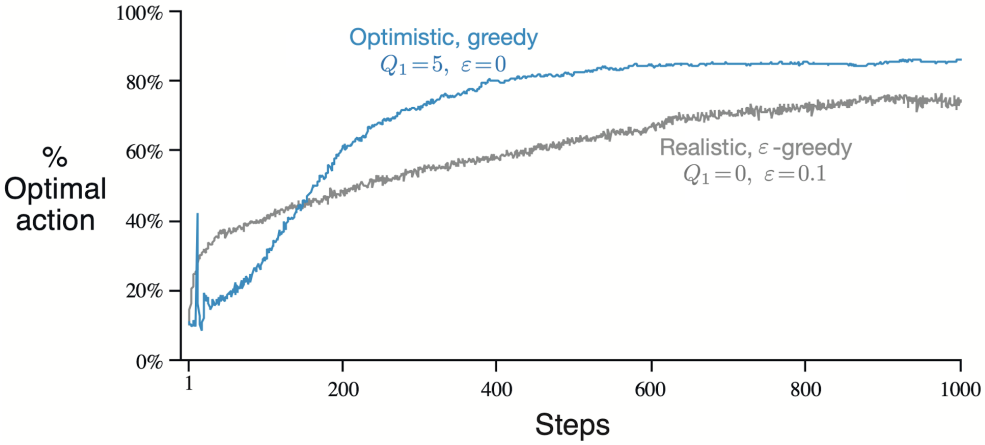


Figure 1.3: Optimistic initial values in action-value methods. The agent starts with high initial estimates for all actions, encouraging exploration until the true values are learned.

where $\pi_t(a)$ indicates the probability of taking action a at time step t. The action probabilities are normalized so that they sum to one.

There is a natural learning algorithm for soft-max action preferences based on the idea of stochastic gradient ascent. On each step, after selecting action A_t and receiving the reward R_t, the action preferences are updated by:

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R}_t)(1 - \pi_t(A_t)), \quad \text{and}$$
$$H_{t+1}(A_t) = H_t(A_t) - \alpha(R_t - \bar{R}_t)\pi_t(a) \quad \text{for all } a \neq A_t$$

where $\bar{R}_t$ is the average reward received up to time step t. This update rule adjusts the preferences based on the received reward, encouraging actions that yield higher rewards and discouraging those that yield lower rewards.

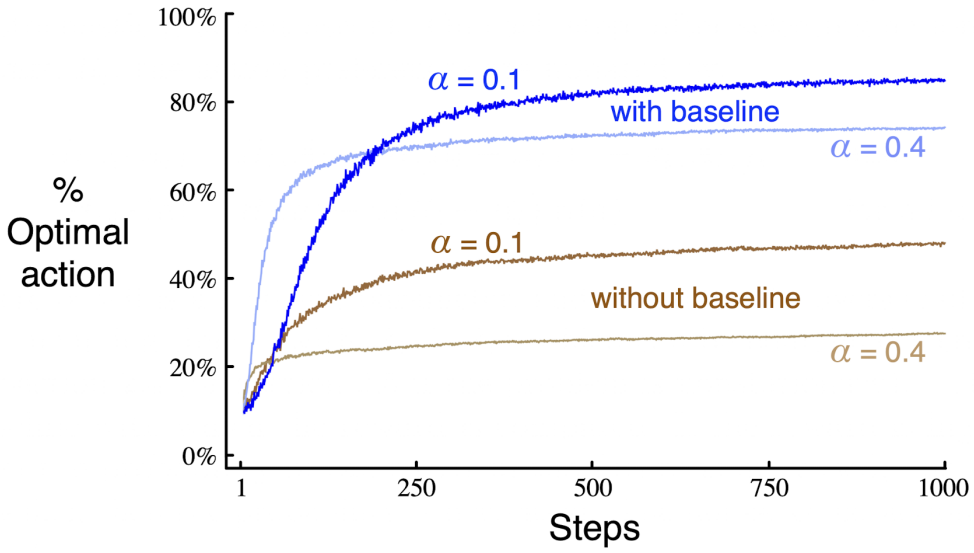


Figure 1.4: Gradient bandit algorithms for action selection. The agent learns preferences for actions based on the received rewards, using a soft-max policy to select actions probabilistically.

Chapter 2

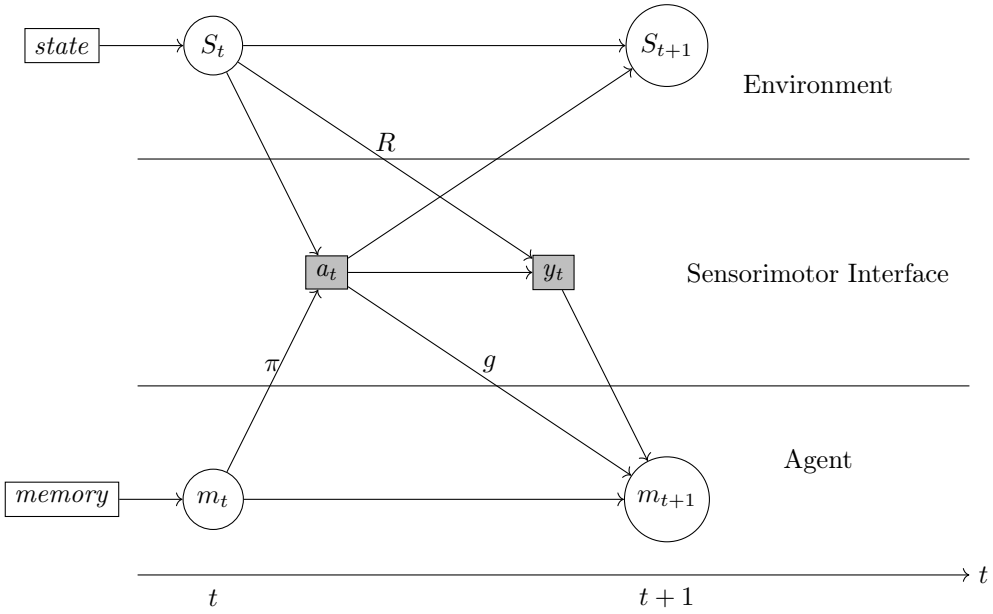
Markov Decision Processes

A Markov Decision Process (MDP) is a classical formalization of sequential decision-making, where actions influence not just immediate rewards but also subsequent states and, through those, future rewards. Whereas in bandit problems we estimated the value $q_*(a)$ of each action a, in MDPs we estimate either $q_*(s, a)$ - the value of each action in each state - or $v_*(s)$ - the value of each state under optimal action selection.

2.1 Interface

The learner and decision-maker is called the **agent**. Everything it interacts with, comprising everything outside the agent, is called the **environment**. These interact continually: the agent selects actions, and the environment responds by presenting new situations and emitting rewards - special numerical values that the agent seeks to maximize over time.

The general scheme of a *Decision Process* is shown below:



The four key components of the interface are:

- $\pi(a|m)$: policy
- $R(y)$: reward function
- $p(s', y|s, a)$: model of the environment (transition and observation)
- $g(m'|m, a, y)$: memory update

Discount factor Furthermore, to incentivize the agent to complete tasks efficiently, we introduce a **discount factor** (or survival probability) γ, with $0 \leq \gamma < 1$. At each time step the agent survives with probability γ and transitions to a terminal absorbing state (a "coffin" state) with probability $1 - \gamma$.

The expected survival time is:

$$T_{\text{eff}} = \frac{1}{1-\gamma}$$

The goal is to find the optimal policy π^* that maximizes the expected discounted return:

$$\pi^* = \arg\max_{\pi} \underbrace{\mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t R(y_t)\right]}_{\text{Expected Return}}$$

The discount factor $\gamma < 1$ ensures the infinite sum converges.

Specifications:

- **Perfect observability** $\rightarrow$ the agent knows the state of the environment $(y = S)$ and $p(y|s, a, s') = \infty(y = s')$

Observation:

$$p(s', y|s, a) = p(s'|s, a)p(y|s, a, s')$$

- **Memory update** $\rightarrow$ the agent knows the state of the environment and the memory $(M = y)$ and $g(m'|m, a, y) = \infty(m' = y)$

At each time step t, the agent receives some representation of the environment's *state* $S_t \in \mathcal{S}$, and on that basis selects an *action* $A_t \in \mathcal{A}(s)$. One time step later, in part as a consequence of its action, the agent receives a numerical *reward* $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$, and finds itself in a new state S_{t+1}.

2.2 Markov Decision Process

At each time step t, the agent receives a representation of the environment's state $S_t \in \mathcal{S}$, selects an action $A_t \in \mathcal{A}(s)$, and receives a scalar reward $R_{t+1} \in \mathcal{R} \subset \mathbb{R}$ while transitioning to a new state S_{t+1}.

A **Markov Decision Process (MDP)** is a fully observable set of tuples (S, A, R, P, γ) where:

- $s \in S$: a finite set of states
- $a \in A$: a finite set of actions
- $R : S \times A \rightarrow \mathbb{R}$: the (expected) reward function
- $P : S \times A \times S \rightarrow [0, 1]$: the transition probability function (model of the environment)
- $\gamma \in [0, 1]$: the discount factor
- $p_0(s)$: the initial state distribution
- $\pi(a|s)$: the policy

In a *finite* MDP, the sets of states, actions, and rewards all have a finite number of elements.

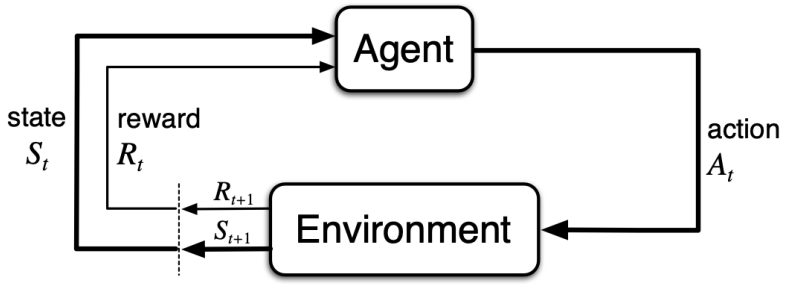


Figure 2.1: Markov Decision Process

2.3 Fully Observable Setting

The most tractable setting is the one with **perfect observability** and **full epistemic knowledge**:

- **Perfect observability**: the agent directly observes the environment's state, i.e., $y_t = S_t$. Formally, $p(y \mid s, a, s') = \mathbf{1}(y = s')$, so the joint model factorizes as:

$$p(s', y \mid s, a) = p(s' \mid s, a) \cdot \mathbf{1}(y = s')$$

Since the observation coincides with the new state, the model simplifies to $p(s' \mid s, a)$.

- **Full epistemic knowledge**: the agent has a correct and complete model of the environment — it knows a priori all transition probabilities $p(s', y \mid s, a)$ and the reward $r'(a, y)$ exactly.
- **Memory update**: since $y_t = s_{t+1}$, the agent simply copies the observation into memory: $m' = y$, i.e., $g(m' \mid m, a, y) = \mathbf{1}(m' = y)$. The memory at initialization is $m_0 = s_0$, and subsequently $m_t = y_{t-1} = s_t$.

In this setting (the top right corner of figure - we see everything, we know everything), the agent does not need to maintain the full history of observations. Knowledge of the current state is sufficient for optimal decision-making. The goal therefore reduces to finding an optimal stationary policy:

$$\pi^*(a \mid s),$$

which is a probabilistic mapping from environment states to actions.

In this case, the random variables R_t and S_t have well defined discrete probability distributions dependent only on the preceding state and action.

$$p(s', r|s, a) = Pr(S_t = s', R_t = r|S_{t-1} = s, A_{t-1} = a)$$

This function defines the dynamics of the MDP, it specifies a probability distribution for each choice of s and a and has four arguments.

$$\sum_{s' \in \mathcal{S}} \sum_{r \in \mathcal{R}} p(s', r|s, a) = 1 \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}$$

The state must include information about all aspects of the past agent-environment interaction that make a difference for the future. If it does, then the state is said to have the *Markov property.*

From the four-argument dynamics function, one can obtain:

- the *state-transition probabilities* $p : \mathcal{S} \times \mathcal{S} \times \mathcal{A} \to [0, 1]$

$$p(s'|s, a) = Pr\{S_t = s'|S_{t-1} = s, A_{t-1} = a\} = \sum_{r \in \mathcal{R}} p(s', r|s, a)$$

- the expected rewards for state-action pairs $r : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$:

$$r(s, a) = \mathbb{E}[R_t|S_{t-1} = s, A_{t-1} = a] = \sum_{r \in \mathcal{R}} r \sum_{s' \in \mathcal{S}} p(s', r|s, a)$$

- the expected rewards for state-action-next-state triples $r : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$:

$$r(s, a, s') = \mathbb{E}[R_t|S_{t-1} = s, A_{t-1} = a, S_t = s'] = \sum_{r \in \mathcal{R}} r \frac{p(s', r|s, a)}{p(s'|s, a)}$$

In general, actions can be any decisions we want to learn how to make, and states can be anything we can know that might be useful in making them. In particular, the boundary between agent and environment is typically not the same as the physical boundary of a robot's or an animal's body. Usually, the boundary is drawn closer to the agent for that.

Generally, anything that cannot be changed arbitrarily by the agent is considered to be outside of it and thus part of its environment. We do not assume that everything in the environment is unknown to the agent.

The MDP framework proposes that whatever the details of the sensory, memory and control apparatus, and whatever objective one is trying to achieve, any problem of learning goal-directed behavior can be reduced to three signals passing back and forth between the agent and its environment:

- **actions:** choices made by the agent
- **states:** basis on which the choices are made
- **rewards:** the agent's goal

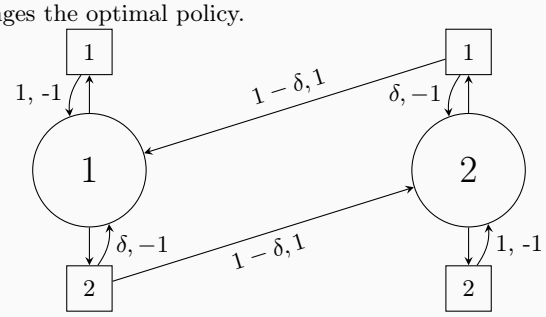
2.4 Partially Observable Markov Decision Process (POMDP)

A **Partially Observable Markov Decision Process (POMDP)** generalizes the MDP framework for situations where the agent cannot directly observe the underlying state s_t. Instead, the agent receives an *observation* y_t that provides partial (and potentially noisy) information about s_t.

In general, because the state is not directly observable, the current observation may not be sufficient for optimal decision-making.

Example: Example: 2-State, 2-Action MDP

Consider a system with two states $\mathcal{S} = \{1, 2\}$ and two actions $\mathcal{A} = \{1, 2\}$. This example illustrates why partial observability fundamentally changes the optimal policy.



The dynamics are defined as follows:

- **State 1:** Action 2 is the "correct" move, transitioning to State 2 with probability $1 - \delta$ (Reward: $+1$). Action 1 is "wrong", keeping the agent in State 1 (Reward: -1).
- **State 2:** Action 1 is the "correct" move, transitioning to State 1 with probability $1 - \delta$ (Reward: $+1$). Action 2 is "wrong", keeping the agent in State 2 (Reward: -1).

In a **fully observable** setting (MDP), the Bellman Optimality Equation for State 1 is:

$$V^*(1) = \max \left\{ \underbrace{-1 + \gamma V^*(1)}_{\text{Action 1}}, \quad \underbrace{(1-\delta)(1 + \gamma V^*(2)) + \delta(-1 + \gamma V^*(1))}_{\text{Action 2}} \right\}$$

Symmetrically, for $V^*(2)$, the optimal policy is deterministic: $\pi^*(1) = 2$ and $\pi^*(2) = 1$.

The POMDP Case: If the agent cannot distinguish between State 1 and State 2 (i.e., observations are uninformative), any **deterministic** policy (e.g., "always take action 2") performs poorly. If the agent is actually in State 2 but thinks it might be in State 1, it will keep receiving $R = -1$ indefinitely. In such cases, a **stochastic policy** (e.g., choosing actions randomly with $p = 0.5$) can be superior, as it guarantees the agent will eventually switch states and collect positive rewards, yielding an expected return higher than the "stuck" deterministic policy.

2.4.1 Observation Model

To formalize the POMDP, we introduce the **observation function** $f(y|s, a, s')$, which defines the probability of receiving observation y given a transition to s'. The joint probability of transitioning and observing is:

$$p(s', y|s, a) = p(s'|s, a) f(y|s, a, s')$$

In a simplified model where the observation depends only on the current state s, we use an error parameter ϵ:

$$f(y|s) = \begin{cases} s, & \text{with probability } 1 - \epsilon \\ \text{noise}, & \text{with probability } \epsilon \end{cases}$$

Analysis of the uncertainty parameter ϵ:

- $\epsilon = 0$: **Full Observability**. The agent knows the state perfectly; the POMDP reduces to an MDP.
- $\epsilon = 0.5$ (in a 2-state system): **Maximum Uncertainty**. The observation is pure noise. The agent has no information about its location, making the problem significantly harder.

2.4.2 Perfect Memory and Information State

In a general POMDP, the agent cannot observe the true state s_t directly, and therefore the current observation y_t is usually not sufficient for optimal decision-making. To act well, the agent must retain information from the past. A natural idea is to let the agent use a **perfect memory**, that is, a memory containing the entire history of actions taken and observations received.

We denote the memory state at time t by m_t. Initially, the memory is empty:

$$m_0 = \emptyset.$$

After that, the memory is updated by concatenating the current action and observation:

$$m_t = a_0 y_1 a_1 y_2 \cdots a_{t-1} y_t.$$

In this setting, the memory contains the whole interaction history. The update rule is therefore deterministic and simply records what happened:

$$g(m' \mid m, a, y) = \mathbf{1}(m' = may).$$

If we consider the pair formed by the hidden state and the memory, namely (s_t, m_t), then the process becomes Markovian in the enlarged space $\mathcal{S} \times \mathcal{M}$. However, the policy still cannot depend directly on the hidden state s_t, because this state is not observable. It can only depend on the memory:

$$\pi(a \mid m).$$

The distribution over hidden states conditioned on the memory,

$$\mu_t(s \mid m),$$

summarizes what the agent knows about the environment after observing the history stored in m. This conditional distribution depends only on the initial distribution and on the sequence of actions and observations contained in the memory. For this reason, the memory can be interpreted as an **information state**: it contains all the information needed to update the agent's knowledge about the hidden state.

This shows that the agent's information, rather than the hidden physical state itself, is what matters for decision-making. In practice, however, working directly with the full history is inconvenient, which motivates the introduction of belief states.

2.4.3 Belief States and the Belief-MDP

Although the full history is sufficient, it is usually inconvenient to work with it directly because its dimension grows with time. A more compact representation is obtained by considering the **belief state**.

A belief state is a probability distribution over the hidden states:

$$b_t(s) = \mathbb{P}(S_t = s \mid \mathcal{H}_t),$$

where $\mathcal{H}_t$ denotes the history of past actions and observations available to the agent up to time t.

Thus, instead of storing the entire history explicitly, the agent stores only its current posterior distribution over states. The belief state is therefore a sufficient statistic for the history: once b_t is known, the full history is no longer needed for decision-making.

The set of all beliefs is the probability simplex over $\mathcal{S}$:

$$b \in \Delta^{|\mathcal{S}|}.$$

When the agent takes action a and then receives observation y, the belief is updated by Bayes' rule. The new belief b' is given by

$$b'(s') = U^{a,y} b(s') = \frac{\sum_s p(s', y \mid s, a)\, b(s)}{\sum_s p(y \mid s, a)\, b(s)}.$$

This update has a simple interpretation:

- the numerator predicts the probability of reaching state s' and observing y after taking action a;
- the denominator normalizes the result and is equal to the probability of observing y under the current belief and action.

More explicitly, the probability of receiving observation y under belief b and action a is

$$P(y \mid a, b) = \sum_s p(y \mid s, a)\, b(s).$$

In this partially observable setting, we write the reward as a function of the observation, consistently with the sensorimotor interface introduced earlier.

The expected immediate reward in belief state b when choosing action a is the average reward over the hidden states weighted by the current belief:

$$r(b, a) = \sum_{s,y} r(y)\, p(y \mid s, a)\, b(s).$$

This allows us to reformulate the POMDP as a standard MDP whose states are beliefs. This is called the **belief-MDP**. Its components are:

- **states:** beliefs $b \in \Delta^{|\mathcal{S}|}$;
- **actions:** the same actions as in the original POMDP;
- **transition dynamics:** belief updates induced by actions and observations;
- **rewards:** expected rewards $r(b, a)$;
- **policy:** a mapping from beliefs to actions, written as $\pi(a \mid b)$.

The key advantage of this formulation is that the problem becomes fully observable again: the agent may not know the true hidden state, but it does know its current belief exactly.

2.4.4 Bellman Equation in Belief Space

Once the POMDP has been rewritten as a belief-MDP, dynamic programming ideas can be applied exactly as in the fully observable case. The only difference is that the state variable is now the belief b rather than the physical state s.

Let $v^*(b)$ denote the optimal value function in belief space. It represents the maximum expected return obtainable when the current belief about the hidden state is b. The Bellman optimality equation becomes

$$v^*(b) = \max_a \left[r(b, a) + \gamma \sum_y P(y \mid a, b)\, v^*(U^{a,y} b) \right].$$

This equation has the same structure as the Bellman equation for ordinary MDPs:

- choose an action a;
- collect the expected immediate reward $r(b, a)$;
- consider all possible observations y that may follow;
- update the belief through Bayes' rule;
- add the discounted value of the updated belief.

The corresponding Bellman operator B acts on functions of the belief state as

$$(Bf)(b) = \max_a \left[r(b, a) + \gamma \sum_y P(y \mid a, b)\, f(U^{a,y} b) \right].$$

The main difficulty is that the belief space is continuous, even when the original state space is finite. For this reason, exact dynamic programming in belief space is often computationally hard, and practical methods usually rely on approximations. Nevertheless, the belief-MDP formulation provides the fundamental theoretical framework for Bayesian decision-making under partial observability.

2.5 Goals and Rewards

At each time step, the reward is a simple number $R_t \in \mathbb{R}$. Informally, the agent's goal is to maximize the total amount of reward it receives. This means maximizing not immediate reward, but cumulative reward in the long run. We can clearly state this informal idea as the *reward hypothesis*:

> *That all of what we mean by goals and purposes can be well thought of as the maximization of the expected value of the cumulative sum of a received scalar signal (called reward).*

In particular, the reward signal is not the place to impart to the agent prior knowledge about *how* to achieve what we want it to do. For example, a chess-playing agent should be rewarded only for actually winning, not for achieving subgoals such as taking its opponent's pieces or gaining control of the center of the board. The reward signal is your way to communicating to the agent *what* you want to be achieved, not *how* you want it to be achieved.

2.6 Returns and Episodes

If the sequence of rewards received after time step t is denoted $R_{t+1}, R_{t+2}, R_{t+3}, \ldots$, then what precise aspect of this sequence do we wish to maximize? In general, we seek to maximize the *expected return*, where the return, denoted as G_{t+1} is defined as some specific function of the reward sequence (most of the cases the sum of the rewards).

This approach makes sense in applications in which there is a natural notion of final time step, i.e., when the agent-environment interaction breaks naturally into sequencces called *episodes*. Each one ends in a special state called

the *terminal state*, followed by a reset to a standard starting state or to a sample from a standard distribution of starting states. The next episode begins independently of how the previous one ended. Tasks with episodes of this kind are called *episodic tasks* and in them we need to distinguish the set of nonterminal states ($\mathcal{S}$), from the set of all states plus the terminal state ($\mathcal{S}^+$).

On the other hand, in many cases the agent-environment interaction does not break naturally into identifiable episodes, but goes on continually without limit. These are the *continuing tasks*. The return formulation is problematic for these because the final time step would be $T = \infty$, such as the the return.

The additional concept that we need is that of *discounting*. According to this approach, the agent tries to select actions so that the sum of the discounted rewards it receives over the future is maximized. In particular, it chooses A_t to maximize the expected *discounted return*:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots = \sum_{k=0}^{\infty} R_{t+k+1},$$

where γ is a parameter defined in $[0, 1]$ called the *discount rate*.

It determines the present value of future rewards: a reward received k time steps in the future is worth only γ^{k-1} times what it would be worth if it were received immediately.

Returns at successive time steps are related to each other in a way that is important for the theory and algorithms of RL:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \dots \\ &= R_{t+1} + \gamma G_{t+1} \end{aligned}$$

where $G_{t+1} = R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4} + \dots$ and if the reward is a constant $+1$, the return is finite

$$G_t = \sum_{k=0}^{\infty} \gamma^k = \frac{1}{1-\gamma}$$

2.7 Unified Notation for Episodic and Continuing Tasks

We number the time steps of each episode starting anew from zero, referring thus not to S_t but to $S_{t,i}$ at time t of episode i. The same for the other parameters. However, it turns out that when we discuss episodic tasks we almost never have to distinguish between different episodes. In practice we almost always abuse notation slightly by dropping the explicit reference to episode number, i.e., we write S_t to refer to $S_{t,i}$ and so on.

The sums for both episodic and continuing tasks can be unified considering episode termination to be the entering of a special *absorbing state* that transitions only to itself and that generates only rewards of zero. Consider the following transition diagram:

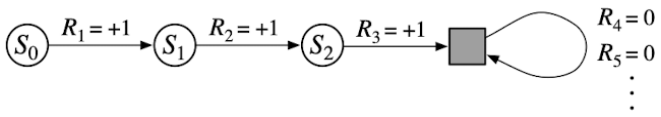


Figure 2.2: Transition diagram for an absorbing state

We can alternatively write

$$G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k,$$

including the possibility that $T = \infty$ or $\gamma = 1$, but not both.

2.8 Policies and Value Functions

Almost all RL algorithms involve estimating *value functions*, i.e., functions of states that estimate how good it is for the agent to be in a given state. The notion of "how good" here is deifned in terms of future rewards that can be expected or in terms of expected return. They are defined with respect to particular ways of acting, called policies. A *policy* is a mapping from states to probabilities of selecting each possible action. If the agent is following policy π at time t, then $\pi(a|s)$ is the probability that $A_t = a$ if $S_t = s$.

The *value function* of a state s under a policy π, denoted $v_\pi(s)$, is the expected return when starting in s and following π thereafter. For MDPs, we can define v_π formally by

$$v_\pi^{(t)}(s) = \mathbb{E}_\pi\left[G_t|S_t = s\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}|S_t = s\right], \text{ for all } s \in \mathcal{S},$$

where $\mathbb{E}_\pi[.]$ denotes the expected value of a random variable given that the agent follows policy π and t is any time step.

We call the function v_π the *state-value function* for policy π.

Similarly, we define the value of taking action a in state s under a policy π, denoted $q_\pi(s,a)$, as the expected return starting from s, taking the action a, and thereafter following policy π:

$$q_\pi(s,a) = \mathbb{E}_\pi\left[G_t|S_t = s, A_t = a\right] = \mathbb{E}_\pi\left[\sum_{k=0}^{\infty}\gamma^k R_{t+k+1}|S_t = s, A_t = a\right].$$

We call q_π the *action-value function* for policy π.

The value functions v_π and q_π can be estimated from experience by averaging the returns G_t observed after each time step t when the agent is following policy π. This leads to the so called *Monte Carlo methods*.

Observation:

A fundamental property of value functions used throughout RL and dynamic programming is that they satisfy recursive relationships similar to that which we have already estabilished for the return. For any policy π and any state s, the following consistency condition holds between the value of s and the value of its possible successor state:

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi\left[G_t|S_t = s\right] \\ &= \mathbb{E}_\pi\left[R_{t+1} + \gamma G_{t+1}|S_t = s\right] \\ &= \textstyle\sum_a \pi(a|s)\sum_{s',r} p(s',r|s,a)\left[r + \gamma v_\pi(s')\right], \text{ for all } s \in \mathcal{S}, \end{aligned}$$

Note how in the last equation we have merged the two sums, one over all the values of s' and the other over all the values of r, into one sum over all possible values of both. For each triple (a, s', r) we compute its probability $\pi(a|s)p(s',r|s,a)$, weight the quantity in brackets by that probability and then sum over all possibilities to get an expected value.

Note that the last equation in the observation is the so called **Bellman equation** for $v\pi$. It expresses a relationship between the value of a state and the values of its successor states, averaging over all the possibilities, weighting each by its probability of occurring.

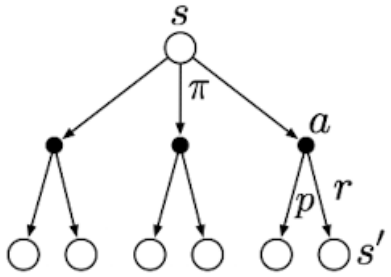


Figure 2.3: Backup diagram for v_π

2.9 Optimal Policies and Value Functions

Solving a RL task means, roughly, finding a policy that achieves a lot of reward over the long run. A policy π is defined to be better than or equal to a policy π' if its expected return is greater than or equal to that of π' for all states. There is always at least one policy that is better than or equal to all other policies, and that is the *optimal policy* π_*. The optimal policies share the same state-value function, called the *optimal state-value function*, denoted v_* and defined as

$$v_*(s) = \max_\pi v_\pi^{(t)}(s), \text{ for all } s \in \mathcal{S}.$$

Optimal policies also share the same *optimal action-value function*, denoted q_* and defined as

$$q_*(s,a) = \max_\pi q_\pi(s,a), \text{ for all } s \in \mathcal{S}, a \in \mathcal{A}(s).$$

Thus, we can write q_* in terms of v_* as follows:

$$q_*(s,a) = \mathbb{E}\left[R_{t+1} + \gamma v_*(S_{t+1}) | S_t = s, A_t = a\right].$$

Because v_* is the value function for a policy, it must satisfy the self-consistency condition given by the Bellman equation for state values. It can be written in a special form without reference to any specific policy. This is the Bellman equation for v_*, or the *Bellman optimality equation*, and expresses the fact that the value of a state under an optimal policy must equal the expected return for the best action from that state.

$$\begin{aligned} v_*^{(t)}(s) &= \max_a q_{\pi_*}(s,a) \\ &= \max_a \mathbb{E}_{\pi_*}\left[G_t | S_t = s, A_t = a\right] \\ &= \max_a \mathbb{E}_{\pi_*}\left[R_{t+1} + \gamma G_{t+1} | S_t = s, A_t = a\right] \\ &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_*^{(t)}(S_{t+1}) | S_t = s, A_t = a\right] \\ &= \max_a \sum_{s',r} p(s', r(s,a,s') | s, a) \left[r + \gamma v_*^{(t)}(s')\right] \end{aligned}$$

The Bellman optimality equation for q_* is

$$\begin{aligned} q_*(s,a) &= \mathbb{E}\left[R_{t+1} + \gamma \max_{a'} q_*(S_{t+1}, a') | S_t = s, A_t = a\right] \\ &= \sum_{s',r} p(s', r | s, a) \left[r + \gamma \max_{a'} q_*(s', a')\right]. \end{aligned}$$

For finite MDPs, the Bellman equations for v_* and q_* have unique solutions. The optimal policies can be derived from these value functions.

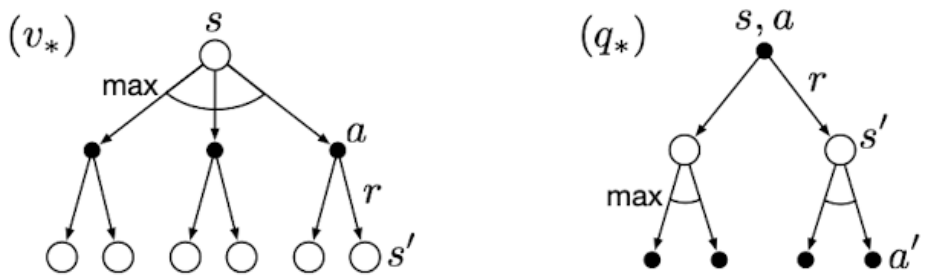


Figure 2.4: Backup diagram for v_* and q_*

Once one has v_*, it is relatively easy to determine an optimal policy. For each state s, there will be one or more actions at which the maximum is obtained in the Bellman optimality equation. Any policy that assigns nonzero probability only to these actions is an optimal policy. Having q_* makes choosing optimal actions even easier. With q_*, the agent does not even have to do a one-step-ahead search: for any state s, it can simply find any action that maximizes $q_*(s,a)$, and that will be an optimal action.

Explicitly solving the Bellman optimality equation provides one route to finding an optimal policy, and thus to solving the RL problem. However, this solution is rarely direcly useful. This solution relies on at least three assumptions that are rarely true in practice:

- the dynamics of the environment are accurately known
- computational resources are sufficient to complete the calculation
- the states have the Markov property

Chapter 3

Dynamic Programming

The term **dynamic programming** (DP) refers to a collection of algorithms that can be used to compute optimal policies when a perfect model of the environment is available and the problem is formulated as a Markov Decision Process (MDP). Classical DP algorithms are of limited practical use in reinforcement learning, both because they assume complete knowledge of the environment and because they can be computationally expensive. Nevertheless, they remain of great theoretical importance.

In this chapter, we usually assume that the environment is a finite MDP. This means that the sets of states, actions, and rewards are finite, and that the dynamics of the environment are described by transition probabilities

$$p(s', r \mid s, a),$$

for all $s \in \mathcal{S}$, $a \in \mathcal{A}$, $r \in \mathcal{R}$, and $s' \in \mathcal{S}$. When the state or action spaces are continuous, a common way to obtain approximate solutions is to discretize them and then apply finite-state DP methods. The central idea of DP is to use **value functions** to organize and guide the search for good policies.

Once the optimal value functions v_* or q_* are known, it is straightforward to derive an optimal policy. These optimal value functions satisfy the **Bellman optimality equations**:

$$v_*(s) = \max_a \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right],$$

$$q_*(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma \max_{a'} q_*(s', a') \right].$$

Equivalently, the action-value function can also be written as

$$q_*(s, a) = \sum_{s',r} p(s', r \mid s, a) \left[r + \gamma v_*(s') \right].$$

As we shall see, DP algorithms are obtained by turning Bellman equations such as these into assignments, that is, into update rules that progressively improve approximations of the desired value functions.

3.1 Policy Evaluation

We begin with the problem of computing the value of a policy π. Given a policy, we want to know how good it is to follow it from each state. This task is called **policy evaluation**.

The quantity we want to compute is the **state-value function**

$$v_\pi(s),$$

which represents the expected return obtained by starting from state s and then following policy π.

Using the definition of return, we can write

$$\begin{aligned} v_\pi(s) &= \mathbb{E}_\pi[G_t \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma G_{t+1} \mid S_t = s] \\ &= \mathbb{E}_\pi[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s]. \end{aligned}$$

If we expand the expectation over all possible actions, next states, and rewards, we obtain the Bellman expectation equation:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big].$$

Here:

- $\pi(a|s)$ is the probability of choosing action a in state s under policy π;
- $p(s', r \mid s, a)$ is the probability of moving to state s' and receiving reward r after taking action a in state s.

If the dynamics of the environment are known, then these equations form a system of $|S|$ linear equations in the $|S|$ unknown values $v_\pi(s)$. In principle, this system can be solved exactly. In practice, however, it is often convenient to use an iterative method.

For this reason, we choose an initial approximation v_0 arbitrarily (with terminal states fixed to 0), and then repeatedly update it according to

$$\begin{aligned} v_{k+1}(s) &= \mathbb{E}[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s] \\ &= \sum_a \pi(a|s) \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_k(s')\big]. \end{aligned}$$

This procedure is called **iterative policy evaluation**. At each step, the old estimate of the value of state s is replaced by a new estimate obtained from:

- the immediate rewards that can be received from s;
- the values of the successor states;
- the probabilities of the transitions and actions under the policy.

This is called an *expected update*, because it uses an expectation over all possible next states and rewards, rather than a single sampled transition.

Under the usual assumptions, the sequence $\{v_k\}$ converges to the true value function v_π as $k \to \infty$.

A complete in-place version of iterative policy evaluation is shown in the pseudocode below.

Iterative Policy Evaluation, for estimating $V \approx v_\pi$

Input π, the policy to be evaluated
Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$ arbitrarily, for $s \in \mathcal{S}$, and $V(\textit{terminal})$ to 0

Loop:
  $\Delta \leftarrow 0$
  Loop for each $s \in \mathcal{S}$:
    $v \leftarrow V(s)$
    $V(s) \leftarrow \sum_a \pi(a|s) \sum_{s',r} p(s', r|s, a)\big[r + \gamma V(s')\big]$
    $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Figure 3.1: Iterative Policy Evaluation Algorithm

3.2 Policy Improvement

Evaluating a policy is useful because it helps us find a better one. Once we know v_π, we can ask whether, in some state s, it would be better to choose a different action than the one currently suggested by π.

To answer this, we introduce the **state-action value function**

$$q_\pi(s, a),$$

which measures the expected return obtained by taking action a in state s first, and then following policy π afterwards.

Formally,

$$\begin{aligned} q_\pi(s, a) &= \mathbb{E}[R_{t+1} + \gamma v_\pi(S_{t+1}) \mid S_t = s, A_t = a] \\ &= \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big]. \end{aligned}$$

This quantity tells us how good action a looks if we evaluate it using the current value function v_π.

Now suppose we define another policy π'. A fundamental result, called the **policy improvement theorem**, states that if for every state s

$$\sum_a \pi'(a|s)\, q_\pi(s,a) \geq v_\pi(s),$$

then the new policy is at least as good as the old one:

$$v_{\pi'}(s) \geq v_\pi(s), \qquad \forall s \in \mathcal{S}.$$

So, if a new policy chooses actions whose expected value under q_π is never worse than the old policy, then the new policy improves on the old one.

The most natural choice is to build a new policy that, in each state, selects the action with the highest $q_\pi(s,a)$. This gives the **greedy policy**:

$$\begin{aligned}\pi'(s) &= \arg\max_a q_\pi(s,a) \\ &= \arg\max_a \sum_{s',r} p(s', r \mid s, a)\big[r + \gamma v_\pi(s')\big].\end{aligned}$$

In deterministic form, this can be written as

$$\pi'(a|s) = \mathbf{1}\Big(a = \arg\max_a q_\pi(s,a)\Big).$$

The greedy policy satisfies the condition of the policy improvement theorem because

$$\sum_a \pi'(a|s) q_\pi(s,a) = \max_a q_\pi(s,a) \geq v_\pi(s).$$

Therefore, the greedy policy is always at least as good as the original one. Moreover, if the greedy policy is different from the old policy, then the improvement is *strict*. This means that changing the policy in this way really produces a better policy, not just an equivalent one.

This step is called **policy improvement**: we use the value function of the current policy to construct a better policy.

3.3 Policy Iteration

Policy iteration is obtained by combining the two previous ideas:

1. evaluate the current policy;
2. improve it by making it greedy with respect to its value function.

Starting from an initial policy π_0, we obtain a sequence

$$\pi_0 \xrightarrow{E} v_{\pi_0} \xrightarrow{I} \pi_1 \xrightarrow{E} v_{\pi_1} \xrightarrow{I} \pi_2 \xrightarrow{E} \cdots$$

where E stands for evaluation and I stands for improvement.

In other words:

- first we compute the value of the current policy;
- then we replace the policy with a greedy one;
- then we evaluate the new policy again;
- and we continue until the policy no longer changes.

The algorithm can be summarized as follows:

initialize $\pi_0,\ k \leftarrow 0$,
repeat
  $v_k = \text{Value}(\pi_k)$
  $\pi_{k+1} = \text{Greedy}(v_k)$
  $k \leftarrow k + 1$
until $\pi_{k+1} = \pi_k$.

A key point is that the stopping criterion is based on the **policy**, not on a numerical tolerance. The algorithm stops when the improvement step no longer changes the policy.

This is enough to guarantee convergence in a finite MDP. Indeed, the number of deterministic policies is finite (although possibly very large), and each genuine improvement step produces a strictly better policy. Therefore, the algorithm cannot continue forever: after a finite number of iterations, it must reach a policy that cannot be improved further.

At convergence, suppose that for some step n we have

$$\pi_{n+1} = \pi_n.$$

Then the corresponding value function v_n satisfies

$$v_n(s) = \max_a \sum_{s'} p(s'|s,a)\big(r(s,a,s') + \gamma v_n(s')\big),$$

which is exactly the **Bellman optimality equation**.

Since this equation has a unique solution, it follows that

$$v_n = v_* \qquad \text{and} \qquad \pi_n = \pi_*.$$

So, the policy obtained when policy iteration stops is an **optimal policy**.

Policy Iteration (using iterative policy evaluation) for estimating $\pi \approx \pi_*$

1. Initialization
   $V(s) \in \mathbb{R}$ and $\pi(s) \in \mathcal{A}(s)$ arbitrarily for all $s \in \mathcal{S}$; $V(\mathit{terminal}) \doteq 0$

2. Policy Evaluation
   Loop:
   $\Delta \leftarrow 0$
   Loop for each $s \in \mathcal{S}$:
   $v \leftarrow V(s)$
   $V(s) \leftarrow \sum_{s',r} p(s', r \,|\, s, \pi(s))\big[r + \gamma V(s')\big]$
   $\Delta \leftarrow \max(\Delta, |v - V(s)|)$
   until $\Delta < \theta$ (a small positive number determining the accuracy of estimation)

3. Policy Improvement
   policy-stable $\leftarrow$ *true*
   For each $s \in \mathcal{S}$:
   old-action $\leftarrow \pi(s)$
   $\pi(s) \leftarrow \arg\max_a \sum_{s',r} p(s', r \,|\, s, a)\big[r + \gamma V(s')\big]$
   If *old-action* $\neq \pi(s)$, then *policy-stable* $\leftarrow$ *false*
   If *policy-stable*, then stop and return $V \approx v_*$ and $\pi \approx \pi_*$; else go to 2

Figure 3.2: Policy Iteration Algorithm

3.4 Comparison with Value Iteration

Let B be the Bellman optimality operator, and assume that both algorithms start from the same initial guess V_0. Then, after the same number of steps k, the value produced by policy iteration is better than the one produced by value iteration:

$$V_k > B^k V_0.$$

The reason is that policy iteration performs a full policy improvement at each step. Before convergence, one has

$$V_k > BV_{k-1},$$

and by repeatedly using the **monotonicity** of the Bellman operator,

$$U \leq W \;\Rightarrow\; BU \leq BW,$$

we obtain

$$V_k > BV_{k-1} > B^2 V_{k-2} > \cdots > B^k V_0.$$

This means that, in terms of number of iterations, policy iteration improves the value estimate at least as fast as value iteration. However, each step of policy iteration is more expensive, because the evaluation step requires solving a linear system or running iterative evaluation. So the trade-off is:

- **policy iteration**: fewer iterations, but heavier steps;
- **value iteration**: lighter steps, but usually more iterations.

Therefore, policy iteration is often faster in terms of convergence in policy space, even if each individual iteration costs more.

3.5 Value Iteration

One drawback of policy iteration is that each of its iterations requires a policy evaluation step, which may itself be a long iterative computation involving several sweeps through the state space. A natural simplification is to truncate the policy evaluation step. An important special case is obtained when policy evaluation is stopped after just one sweep. This gives the algorithm called **value iteration**.

Value iteration can be interpreted as combining, in a single update, one step of policy improvement and one step of truncated policy evaluation. Its update rule is

$$\begin{aligned} v_{k+1}(s) &= \max_a \mathbb{E}\left[R_{t+1} + \gamma v_k(S_{t+1}) \mid S_t = s, A_t = a\right] \\ &= \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma v_k(s')\right]. \end{aligned}$$

It is often convenient to define the **Bellman optimality operator** B as

$$(BW)(s) = \max_a \sum_{s'} p(s' \mid s, a)\left[r(s, a, s') + \gamma W(s')\right].$$

With this notation, value iteration can be written compactly as

$$v_{k+1} = Bv_k.$$

The optimal value function v_* is the unique fixed point of this operator, that is,

$$Bv_* = v_*.$$

Therefore, value iteration can be viewed as a sequence of successive approximations converging to v_*.

Formally, exact convergence to v_* requires infinitely many iterations. In practice, the algorithm is stopped when two successive value functions differ by less than a prescribed tolerance in the max norm:

$$\|v_{k+1} - v_k\|_\infty < \delta.$$

A key property of the Bellman optimality operator is that it is a **contraction** in the max norm, with contraction factor γ:

$$\|BU - BW\|_\infty \le \gamma \|U - W\|_\infty.$$

As a consequence,

$$\|v_{k+1} - v_*\|_\infty = \|Bv_k - Bv_*\|_\infty \le \gamma \|v_k - v_*\|_\infty,$$

which shows that value iteration converges linearly to the optimal value function.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\textit{terminal}) = 0$

Loop:
| $\Delta \leftarrow 0$
| Loop for each $s \in \mathcal{S}$:
| $\quad v \leftarrow V(s)$
| $\quad V(s) \leftarrow \max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$
| $\quad \Delta \leftarrow \max(\Delta, |v - V(s)|)$
until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that
$\pi(s) = \arg\max_a \sum_{s',r} p(s', r \mid s, a)\left[r + \gamma V(s')\right]$

Figure 3.3: Value Iteration Algorithm

Thus, value iteration combines policy improvement and truncated policy evaluation into a single repeated update. Compared with policy iteration, each iteration is simpler and cheaper, although more iterations may be needed. On the other hand, faster convergence is often achieved by performing more than one policy evaluation sweep before each policy improvement step, leading back toward policy iteration or intermediate methods between the two.

3.6 Asynchronous DP

A major drawback to the DP methods that we have discussed so far is that they involve operations over the entire state set of the MDP, i.e., they require sweeps of the state set. If the state set is very large, then even a single sweep can be prohibitively expensive.

Asynchronous DP algorithms are in-place iterative DP algorithms that are not organized in terms of systematic sweeps of the state set. These algorithms update the values of states in any order whatsoever, using whatever values of other states happen to be available. The values of some states may be updated several times before the values of others are updated once. To converge correctly, however, an asynchronous algorithm must continue to update the values of all the states: it cannot ignore any state after some point in the computation. Asynchronous DP algorithms allow great flexibility in selecting states to update.

Avoiding sweeps does not necessarily mean that we can get away with less computation. It just means that an algorithm does not need to get locked into any hopelessly long sweep before it can make progress improving a policy. We can try to take advantage of this flexibility by selecting the states to which we apply updates so as to improve the algorithm's rate of progress. We can try to order the updates to let value information propagate from state to state in an efficient way. Some states may not need their values updated as often as others.

3.7 Generalized Policy Iteration

Policy iteration consists of two simultaneous, interacting processes: one making the value function consistent with the current policy (policy evaluation), and the other making the policy greedy with respect to the current value function (policy improvement). In policy iteration, these two processes alternate, each completing before the other begins, even though this is not really necessary.

We use the term **generalized policy iteration (GPI)** to refer to the general idea of letting policy-evaluation and policy-improvement processes interact, independent of the granularity and other details of the two processes. Almost all RL methods are well described as GPI. That is, all have identifiable policies and value functions, with the policy always being improved with respect to the value function for the policy. If both the evaluation process and the improvement process stabilize, then the value function and policy must be optimal.

3.8 Linear Programming for MDP

Primal: optimize a function $f(u)$ where $u \in \mathbb{R}^{|S|}$

$$f(u) = \sum_s p_0(s)u(s)$$

$$\Omega = \left\{ u \in \mathbb{R}^{|S|} : \sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma u(s')\big) - u(s) \leq 0 \quad \forall s \in S,\ \forall a \in A \right\}$$

Ω is a polyhedron and a convex set. The objective is to minimize the function $f(u)$ over the set Ω.

Observation:

If $u \in \Omega$ we call it a **Feasible Point**.

The solution of the primal MDP linear programming = optimal solution of the MDP

$$\max_\pi G_\pi(p_0) = f(V^*) = \min_{u \in \Omega} f(u)$$

- V^* is feasible: $V^* \in \Omega \Rightarrow f(V^*) \geq \min_{u\in\Omega} f(u)$.
  Since
  $$\sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma V^*(s')\big) \leq \max_a \left[\sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma V^*(s')\big) \right] = V^*(s) \quad \forall s, a$$

- If u is feasible, then $u \geq V^*$ component-wise (meaning $\forall s,\ u(s) \geq V^*(s)$).

$$\text{if } u \in \Omega,\ \text{choose } a^*(s) = \arg\max_a \left[\sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma V^*(s')\big) \right]$$

$$\begin{aligned}
&u(s) - V^*(s) \\
&\geq \sum_{s'} p\big(s' \mid sa^*(s)\big)\big(r(sa^*(s)s') + \gamma u(s')\big) - \max_a \left[\sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma V^*(s')\big) \right] \\
&= \sum_{s'} p\big(s' \mid sa^*(s)\big)\big(r(sa^*(s)s') + \gamma u(s')\big) - \sum_{s'} p\big(s' \mid sa^*(s)\big)\big(r(sa^*(s)s') + \gamma V^*(s')\big) \\
&= \gamma \sum_{s'} p\big(s' \mid sa^*(s)\big)\big(u(s') - V^*(s')\big) \\
&\geq \gamma \min_{s'}\big(u(s') - V^*(s')\big)
\end{aligned}$$

Taking the minimum over s on both sides:

$$(1-\gamma)\min_s\big(u(s) - V^*(s)\big) \geq 0$$

hence $u(s) \geq V^*(s)$ for all s, and therefore

$$f(u) \geq f(V^*)$$

Primal problem:

$$\min_{u \in \Omega} f(u)$$

Can we go from this constrained problem to an unconstrained problem? Yes.

$$\min_{u \in \Omega} f(u) = \min_{u \in \mathbb{R}^{|S|}} [f(u) + h(u)]$$

with

$$h(u) = \begin{cases} +\infty & \text{if } u \notin \Omega \\ 0 & \text{if } u \in \Omega \end{cases}$$

$$h(u) = \max_{z \geq 0} \left[\sum_{sa} z(sa) \left(\sum_{s'} p(s' \mid sa)\big(r(sas') + \gamma u(s')\big) - u(s) \right) \right]$$

thus:

$$\min_{u \in \Omega} f(u) = \min_u \max_{z \geq 0} \mathbb{L}(u, z)$$

$$\mathbb{L}(u,z) = \sum_{sas'} z(sa)p(s' \mid sa)r(sas') + \sum_{s'} u(s') \left(p_0(s') + \gamma \sum_{sa} p(s' \mid sa)z(sa) - \sum_{a'} z(s'a') \right)$$

Let us now consider the problem that arises if we swap the minimum and the maximum in the saddle-point problem:

$$\max_{z \geq 0} \min_u \mathbb{L}(u, z)$$

Defining

$$g(z) = \sum_{sas'} z(sa)p(s' \mid sa)r(sas')$$

we have

$$\min_u \mathbb{L}(u,z) = g(z) + \min_u \left[\sum_{s'} u(s') \left(p_0(s') + \gamma \sum_{sa} p(s' \mid sa)z(sa) - \sum_{a'} z(s'a') \right) \right]$$

The second term takes value $-\infty$ unless all terms in brackets vanish. Therefore we can rephrase the max-min saddle problem as a constrained maximization problem:

$$\max_{z \geq 0} \min_u \mathbb{L}(u, z) = \max_{z \in \Theta} g(z)$$

where

$$\Theta = \left\{ z \in \mathbb{R}^{|S||A|} : z(sa) \geq 0,\ p_0(s') + \gamma \sum_{sa} p(s' \mid sa) z(sa) - \sum_{a'} z(s'a') = 0 \quad \forall s' \in S \right\}$$

The optimization problem

$$\max_{z \in \Theta} g(z)$$

is the **dual MDP linear program**.

$$\min_{u \in \Omega} f(u) = \max_{z \in \Theta} g(z)$$

First, it is straightforward to show that

$$\min_{u \in \Omega} f(u) = \min_{u} \max_{z \geq 0} \mathbb{L}(u, z) \geq \max_{z \geq 0} \min_{u} \mathbb{L}(u, z) = \max_{z \in \Theta} g(z)$$

To prove the converse, let us introduce the expected discounted occupancy of the state-action pair sa under a given policy π:

$$\eta^{\pi}(sa) \equiv \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t \mathbf{1}(S_t = s, A_t = a) \right] = \sum_{t=0}^{\infty} \gamma^t \rho_t(s) \pi(a \mid s)$$

and remark that the return of policy π can be written as

$$G^{\pi}(p_0) = \sum_{sas'} \eta^{\pi}(sa) p(s' \mid sa) r(sas') = g(\eta^{\pi})$$

The key point here is to recognize that $\eta^{\pi} \in \Theta$ for any policy π. Indeed,

$$\eta^{\pi}(sa) \geq 0 \qquad \forall s, a$$

and from the Kolmogorov-Chapman equations

$$\sum_{a'} \eta^{\pi}(s'a') = p_0(s') + \gamma \sum_{sa} \eta^{\pi}(sa) p(s' \mid sa)$$

In particular, for the optimal policy π^* we have

$$\min_{u \in \Omega} f(u) = G^{\pi^*}(p_0) = g(\eta^{\pi^*}) \leq \max_{z \in \Theta} g(z)$$

which concludes the proof.
This result also shows that

$$z^* = \arg\max_{z \in \Theta} g(z) = \eta^{\pi^*}$$

and therefore the optimal policy can be read from z^* as follows:

$$\pi^*(a \mid s) = \frac{\eta^{\pi^*}(sa)}{\sum_a \eta^{\pi^*}(sa)} = \frac{z^*(sa)}{\sum_a z^*(sa)}$$

If the denominator is zero for some state s, so is the numerator and the choice of the optimal policy is arbitrary. This has no consequences because if

$$\sum_{a} \eta^{\pi^*}(sa) = 0$$

then the state s is never visited under the optimal policy and so it is irrelevant.

Chapter 4

Monte Carlo Methods

Here we do not assume complete knowledge of the environment. Monte Carlo methods require only **experience**, meaning sample sequences of states, actions and rewards from actual or simulated interaction with an environment. Learning from *actual* experience is striking because it requires no prior knowledge of the environment's dynamics, yet can still attain optimal behavior. Learning from *simulated* experience is also powerful. Altough a morel is required, the model need only generate sample transitions, not the complete probability distributions of all possible transitions that is required for dynamic programming (DP).

Monte Carlo methods (MC) are ways of solving the RL problem based on averaging sample returns. Here are defined only for episodic tasks. Only on the completion of an episode are value estimates and policies changed.

Essentially, MC methods sample and average *returns* for each state-action pair much like the bandit methods which sample and average rewards for each action. The main difference is that now there are multiple states, each acting like a different bandit problem and the different bandit problems are interrelated. That is, the return after taking an action in one state depends on the actions taken in later states in the same episode. Because all the action selections are undergoing learning, the problem becomes nonstationary from the point of view of the earlier state.

To handle the nonstationarity, we adapt the idea of general policy iteration (GPI) developed earlier for DP. Whereas there we computed value functions from knowledge of the MDP, here we *learn* value functions from sample returns with the MDP.

4.1 Monte Carlo Prediction

We begin by considering MC methods for learning the state-value function for a given policy. An obvious way to estimate it from experience is simply to average the returns observed after visits to that state. As more returns are observed, the average should converge to the expected value. Suppose we wish to estimate $v_\pi(s)$, the value of a state s under policy π given a set of episodes obtained by following π and passing through s. Each occurrence of state s in an episode is called a *visit* to s. The *first-visit MC method* estimates $v_\pi(s)$ as the average of the returns following the first visits to s, whereas the *every-visit MC method* averages the returns following all visits to s.

First-visit MC prediction, for estimating $V \approx v_\pi$

Input: a policy π to be evaluated
Initialize:
    $V(s) \in \mathbb{R}$, arbitrarily, for all $s \in \mathcal{S}$
    $Returns(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$

Loop forever (for each episode):
    Generate an episode following π: $S_0, A_0, R_1, S_1, A_1, R_2, \ldots, S_{T-1}, A_{T-1}, R_T$
    $G \leftarrow 0$
    Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
        $G \leftarrow \gamma G + R_{t+1}$
        Unless S_t appears in $S_0, S_1, \ldots, S_{t-1}$:
            Append G to $Returns(S_t)$
            $V(S_t) \leftarrow$ average($Returns(S_t)$)

Both first-visit and every-visit MC converge to $v_\pi(s)$ as the number of visits to s goes to infinity.

We can extend the idea of backup diagrams to Monte Carlo algorithms. In general, a backup diagram represents

the quantity being updated as the root node at the top, and shows below it the transitions, rewards, and leaf nodes whose estimated values contribute to that update.

For Monte Carlo estimation of (v_π), the root is a state node. Below it, the diagram contains the entire trajectory generated in a single episode, from that state until the terminal state. The return observed along this complete episode is then used to update the value estimate of the starting state.

A key property of Monte Carlo methods is that state-value estimates are independent in the sense that they do not bootstrap from estimates of successor states. Unlike dynamic programming or temporal-difference methods, Monte Carlo methods wait until the end of an episode and use the actual sampled return.

This also means that the computational cost of estimating the value of a single state does not directly depend on the total number of states in the state space. As a result, Monte Carlo methods can be particularly useful when we are interested only in the value of one state or a small subset of states. In that case, we can generate many sample episodes starting from the states of interest, average the observed returns from those episodes, and ignore the remaining states.

4.2 Monte Carlo Estimation of Action Values

If a model is not available, it is often more useful to estimate action values rather than state values. When a model is available, state values are sufficient to determine a policy: one can look ahead one step, evaluate the immediate reward and the value of the resulting next state, and choose the action that gives the best expected outcome. Without a model, however, state values alone are not enough, because the agent cannot predict the consequences of each available action. Therefore, it must explicitly estimate the value of each state-action pair.

The policy evaluation problem for action values consists of estimating ($q_\pi(s, a)$), the expected return obtained by starting in state (s), taking action (a), and then following policy (π) thereafter. A state-action pair ((s,a)) is said to be visited in an episode if, at some point during that episode, the agent visits state (s) and selects action (a).

The main complication is that many state-action pairs may never be visited. For example, if (π) is a deterministic policy, then in each state the agent will always choose the same action. As a result, returns will only be observed for that action, while the other possible actions from the same state will not receive any updates. This is a serious issue because the purpose of learning action values is precisely to compare the actions available in each state and choose among them. This is known as the problem of **maintaining exploration**.

For Monte Carlo policy evaluation of action values to work properly, we must ensure continual exploration. One way to do this is to assume that each episode starts from a state-action pair, and that every state-action pair has a nonzero probability of being selected as the starting point. Under this assumption, all state-action pairs will be visited infinitely often in the limit of infinitely many episodes. This is called the assumption of **exploring starts**.

4.3 Monte Carlo Control

Let's consider a MC version of classical policy iteration. In this method, we perform alternating complete steps of policy evaluaton and policy improvement, beginning with an arbitrary policy π_0 and ending with the optimal policy and optimal action-value function:

$$\pi_0 \to^E q_{\pi_0} \to^I \pi_1 \to^E q_{\pi_1} \to^I \pi_2 \to^E \dots \to^I \pi_* \to^E q_*,$$

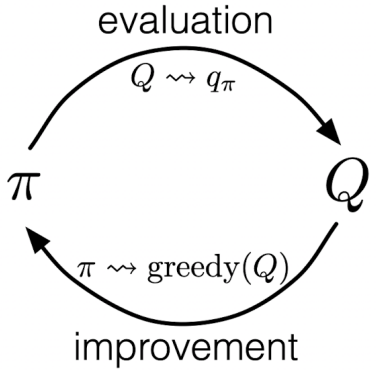
where $\to^E$ denotes a complete policy evaluation and $\to^I$ denotes a complete policy improvement.

Policy evaluation is done exactly as described in the preceding section. Many episodes are experienced, with the approximate action-value function approaching the true function asymptotically. For the moment, let us assume that we do indeed observe an infinite number of episodes and that, in addition, the episodes are generated with exploring starts. Under these assumptions, the MC methods will compute each q_{π_k} exactly, for arbitrary π_k.

Policy improvement is done by making the policy greedy with respect to the current value function. In this case we have an action-value function, and therefore no model is needed to construct the greedy policy. For any given action-value function q, the corresponding greedy policy is the one that, for each $s \in \mathcal{S}$, deterministically chooses an action with maximal action-value:

$$\pi(s) = \arg\max aq(s, a)$$

Policy improvement then can be done by constructing each π_{k+1} as the greedy policy with respect to q_{π_k}.



The first strong assumption we made (exploring starts) is easy to remove. In both DP and MC cases there are two ways to solve the problem. One is to hold firm to the idea of approximating q_{π_k} in each policy evaluation. Measurements and assumptions are made to obtain bounds on the magnitude and probability of error in the estimates, and then sufficient steps are taken during each policy evaluation to assure that these bounds are sufficiently small.

The second approach is to give up trying to complete policy evaluation before returning to policy improvement. On each evaluation step we move the value function toward q_{π_k}, but we don't expect to actually get close except over many steps. Below is the pseudocode of the so called **Monte Carlo with Exploring Starts (MCES)**.

Monte Carlo ES (Exploring Starts), for estimating $\pi \approx \pi_*$

Initialize:
  $\pi(s) \in \mathcal{A}(s)$ (arbitrarily), for all $s \in \mathcal{S}$
  $Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$
  $Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Loop forever (for each episode):
  Choose $S_0 \in \mathcal{S}$, $A_0 \in \mathcal{A}(S_0)$ randomly such that all pairs have probability > 0
  Generate an episode from S_0, A_0, following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
  $G \leftarrow 0$
  Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
    $G \leftarrow \gamma G + R_{t+1}$
    Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:
      Append G to $Returns(S_t, A_t)$
      $Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$
      $\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$

Figure 4.1: Monte Carlo with Exploring Starts (MCES)

4.4 Monte Carlo Control without Exploring Starts

How can we avoid the unlikely assumption of exploring starts? The only general way to ensure that all actions are selected infinitely often is for the agent to continue to select them. There are two approaches:

- **On-policy methods** - attempt to evaluate or improve the policy that is used to make decisions
- **Off-policy methods** - evaluate or improve a policy different from the one used to generate the data

In on-policy control methods, the policy is generally *soft*, meaning that $\pi(a|s) > 0$ for all $s \in \mathcal{S}$ and all $a \in \mathcal{A}(s)$, but gradually shifted closer and closer to a deterministic optimal policy.

The on-policy control method we present in this section uses ϵ-greedy policies, meaning that most of the time they choose an action that has maximal estimated action value, but with probability ϵ they instead select an action at random.

As in MCES, we use first-visit MC methods to estimate the action-value function for the current policy. Without the assumption of exploring starts, however, we cannot simply improve the policy by making it greedy with respect to the current value function, because that would prevent further exploration of nongreedy actions. Fortunately, GPI does not require that the policy be taken all the way to a greedy policy, only that it be moved toward a greedy policy. In our on-policy method we will move it only to an ϵ-greedy policy. For any ϵ-soft policy, π, any ϵ-greedy policy with respect to q_π is guaranteed to be better than or equal to π.

On-policy first-visit MC control (for ε-soft policies), estimates $\pi \approx \pi_*$

Algorithm parameter: small $\varepsilon > 0$

Initialize:

$\pi \leftarrow$ an arbitrary ε-soft policy

$Q(s,a) \in \mathbb{R}$ (arbitrarily), for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

$Returns(s,a) \leftarrow$ empty list, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$

Repeat forever (for each episode):

Generate an episode following π: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$

$G \leftarrow 0$

Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:

$G \leftarrow \gamma G + R_{t+1}$

Unless the pair S_t, A_t appears in $S_0, A_0, S_1, A_1 \ldots, S_{t-1}, A_{t-1}$:

Append G to $Returns(S_t, A_t)$

$Q(S_t, A_t) \leftarrow \text{average}(Returns(S_t, A_t))$

$A^* \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken arbitrarily)

For all $a \in \mathcal{A}(S_t)$:

$$\pi(a|S_t) \leftarrow \begin{cases} 1 - \varepsilon + \varepsilon/|\mathcal{A}(S_t)| & \text{if } a = A^* \\ \varepsilon/|\mathcal{A}(S_t)| & \text{if } a \neq A^* \end{cases}$$

Figure 4.2: Monte Carlo Control with ϵ-greedy policies

So, the policy ieration workds for ϵ-soft policies. Using the natural notion of greedy policy for ϵ-soft policies, one is assured of improvement on every step, except when the best policy has been found among the ϵ-soft policies. This analysis is independent of how the action-value functions are determined at each stage, but it does assume that they are computed exactly. Now we only achieve the best policy among the ϵ-soft policies, but on the other hand, we have eliminated the assumption of exploring starts.

4.5 Off-policy Prediction via Importance Sampling

All learning control methods face a dilemma: they seek to learn action values conditional on subsequent optimal behavior, but they need to behave non-optimally in order to explore all actions. How can they learn about the optimal policy while behaving according to an exploratory policy? The on-policy approach in the preceding section is actually a compromise, i.e., it learns action values not for the optimal policy but for a near-optimal policy that still explores. A more straightforward approach is to use two policies, one that is learned about and that becomes the optimal policy, and one that is more exploratory and is used to generate behavior. The first is the **target policy**, while the second is the **behavior policy**. In this case we say that learning is from data "off" the target policy, and the overall process is termed *off-policy learning*.

Off-policy methods are often of greater variance and are slower to converge. On the other hand, off-policy methods are more powerful in general.

We begin by considering the *prediction* problem, where both target and behavior policies are fixed. Suppose we wish to estimate v_π or q_π, but all we have are episodes following another policy b, where $b \neq \pi$. In this case, π is the target policy, b is the behavior policy, and both policies are considered fixed and given.

We require that every action taken under π is also taken, at least occasionally, under b. This is called the assumption of *coverage*. It follows from coverage that b must be stochastic in states where it is not identical to π. The target policy π, on the other hand, may be deterministic.

Almost all off-policy methods utilize **importance sampling**, a general technique for estimating expected values under one distribution given samples from another. We apply importance sampling to off-policy learning by weighting returns according to the relative probability of their trajectories occurring under the target and behavior policies, called the *importance-sampling ratio*. Given a starting state S_t, the probability of the subsequent state-action trajectory occurring under any policy π is:

$$\begin{aligned} &Pr\{A_t, S_{t+1}, A_{t+1}, \ldots, S_T | S_t, A_{t:T-1} \approx \pi\} \\ &\quad = \pi(A_t|S_t) p(S_{t+1}|S_t, A_t) \pi(A_{t+1}|S_{t+1}) \ldots p(S_T|S_{T-1}, A_{T-1}) \\ &\quad = \prod_{k=t}^{T-1} \pi(A_k|S_k) p(S_{k+1}|S_k, A_k) \end{aligned}$$

where p here is the state-transition probability function.

The importance-sampling ratio is:

$$\rho_{t:T-1} = \frac{\prod_{k=T}^{T-1} \pi(A_k|S_k)p(S_{k+1}|S_k, A_k)}{\prod_{k=t}^{T-1} b(A_k|S_k)p(S_{k+1}|S_k, A_k)} = \prod_{k=t}^{T-1} \frac{\pi(A_k|S_k)}{b(A_k|S_k)}$$

Recall that we wish to estimate the expected returns (values) under the target policy, but all we have are returns G_t due to the behavior policy. These returns have the wrong expectation $\mathbb{E}[G_t|S_t = s] = v_b(s)$ and so cannot be averaged to obtain v_π This is where importance sampling comes in. The ratio $\rho_{t:T-1}$ transforms the returns to have the right expected value:

$$\mathbb{E}[\rho_{t:T-1}G_t|S_t = s] = v_\pi(s)$$

We can define the set of all time steps in which state s is visited, denoted $\mathcal{T}(s)$. Also, let $T(t)$ denote the first time of termination following time t, and G_t denote the return after t up through $T(t)$. Then $\{G_t\}_{t\in\mathcal{T}(s)}$ are the returns that pertain to state s, and $\left\{\rho_{t:T(t)-1}\right\}_{t\in\mathcal{T}(s)}$ are the corresponding importance-sampling rations. To estimate $v_\pi(s)$, we simply scale the returns by the ratios and average the results:

$$V(s) = \frac{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1}G_t}{|\mathcal{T}(s)|}$$

When importance sampling is done as a simple average in this way it is called *ordinary importance sampling*.

An important alternative is *weighted importance sampling*, which uses a weighted average, defined as:

$$V(s) = \frac{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1}G_t}{\sum_{t\in\mathcal{T}(s)} \rho_{t:T(t)-1}}$$

4.6 Incremental Implementation

MC prediction methods can be implemented incrementally, on an episode-by-episode basis, using extensions of the techniques described in Chapter 3. In MC methods we average returns. In all other aspects the same methods of Chapter 3 can be used for on-policy MC methods. For off-policy MC methods, we need to separately consider those that use ordinary importance sampling and those that use weighted importance sampling.

In ordinary importance sampling, the returns are scaled by the importance sampling ratio $\rho_{t:T(t)-1}$, then simply averaged. This leaves the case of off-policy methods using weighted importance sampling. Here we have to form a weighted average of the returns, and a slightly different incremental algorithm is required. We omit the full derivation here.

4.7 Off-policy Monte Carlo Control

Recall that the distinguishing feature of on-policy methods is that they estimate the value of a policy while using it for control. In off-policy methods these two functions are separated. The policy used to generate behavior, called the *behavior policy*, may in fact be unrelated to the policy that is evaluated and improved, called the *target policy*. An advantage of this separation is that the target policy may be deterministic (e.g. greedy), while the behavior policy can continue to sample all possible actions.

These techniques require that the behavior policy has a nonzero probability of selecting all actions that might be selected by the target policy (coverage). To explore all possibilities, we require that the behavior policy be soft.

Off-policy MC control, for estimating $\pi \approx \pi_*$

Initialize, for all $s \in \mathcal{S}$, $a \in \mathcal{A}(s)$:
$Q(s,a) \in \mathbb{R}$ (arbitrarily)
$C(s,a) \leftarrow 0$
$\pi(s) \leftarrow \arg\max_a Q(s,a)$ (with ties broken consistently)

Loop forever (for each episode):
$b \leftarrow$ any soft policy
Generate an episode using b: $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$
$G \leftarrow 0$
$W \leftarrow 1$
Loop for each step of episode, $t = T-1, T-2, \ldots, 0$:
$G \leftarrow \gamma G + R_{t+1}$
$C(S_t, A_t) \leftarrow C(S_t, A_t) + W$
$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \frac{W}{C(S_t, A_t)} [G - Q(S_t, A_t)]$
$\pi(S_t) \leftarrow \arg\max_a Q(S_t, a)$ (with ties broken consistently)
If $A_t \neq \pi(S_t)$ then exit inner Loop (proceed to next episode)
$W \leftarrow W \frac{1}{b(A_t|S_t)}$

Figure 4.3: Off-policy Monte Carlo Control

A potential problem is that this method learns only from the tails of episodes, when all of the remaining actions in the episode are greedy. If nongreedy actions are common, then learning will be slow, particularly for states appearing in the early portions of long episodes.

Chapter 5

Temporal-Difference Learning

TD learning is a combination of Monte Carlo ideas and dynamic programming (DP) ideas. Like Monte Carlo methods, TD methods can learn directly from raw experience without a model of the environment's dynamics. Like DP, TD methods update estimates based in part on other learned estimates, without waiting for a final outcome (they bootstrap).

As usual, we start by focusing on the policy evaluation or *prediction* problem, the problem of estimating the value function v_π for a given policy π. For the *control* problem, DP, TD and Monte Carlo methods all use some variation of generalized policy iteration (GPI).

5.1 TD Prediction

Both TD and Monte Carlo methods use experience to solve the prediction problem. Given some experience following a policy π, both methods update their estimate V of v_π for the nonterminal states S_t occurring in that experience.

$$V(S_t) \leftarrow V(S_t) + \alpha \left[G_t - V(S_t)\right] \tag{5.1}$$

Whereas Monte Carlo methods must wait until the end of the episode to determine the increment to $V(S_t)$, TD methods need to wait only until the next time step. At time $t+1$ they immediately form a target and make a useful update using the observed reward R_{t+1} and the estimate $V(S_{t+1})$. The simplest TD method makes the update

$$V(S_t) \leftarrow V(S_t) + \alpha \left[R_{t+1} + \gamma V(S_{t+1} - V(S_t))\right] \tag{5.2}$$

This TD method is called *one-step TD (TD(0))*.

Tabular TD(0) for estimating v_π

Input: the policy π to be evaluated
Algorithm parameter: step size $\alpha \in (0, 1]$
Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\textit{terminal}) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    $A \leftarrow$ action given by π for S
    Take action A, observe R, S'
    $V(S) \leftarrow V(S) + \alpha\big[R + \gamma V(S') - V(S)\big]$
    $S \leftarrow S'$
  until S is terminal

Figure 5.1: Pseudocode for one-step TD learning.

Because TD(0) bases its update in part on an existing estimate, we say that it is a *bootstrapping* method, like DP.

$$v_\pi(s) = \mathbb{E}_\pi [G_t | S_t = s] \tag{5.3}$$
$$= \mathbb{E}_\pi [R_{t+1} + \gamma G_{t+1} | S_t = s]$$
$$= \mathbb{E}_\pi [R_{t+1} + \gamma v_\pi(S_{t+1}) | S_t = s] \tag{5.4}$$

Monte Carlo methods use an estimate of (5.3) as a target, whereas DP methods use an estimate of (5.4) as a target. The Monte Carlo target is an estimate because the expected value in (5.3) is not known; a sample return is used in place of the real expected return. The DP target is an estimate not because of the expected values, which are assumed to be completely provided by a model of the environment, but because $v_\pi(S_{t+1})$ is not known and the current estimate, $V(S_{t+1})$, is used instead. The TD target is an estimate for both reasons: it samples the expected values in (5.4) and it uses the current estimate V instead of the true v_π. Thus, TD methods combine the sampling Monte Carlo with the bootstrapping of DP.

We refer to TD and Monte Carlo updates as *sample updates* because they involve looking ahead to a sample successor state (or state-action pair), using the value of the successor and the reward along the way to compute a backed-up value, and then updating the value of the original state (or state-action pair) accordingly. *Sample* updates differ from the *expected* updates of DP methods in that they are based on a single sample successor rather than on a complete distribution of all possible successors.

Finally, note that the quantity in brackets in $TD(0)$ update is a sort of error, measuring the difference between the estimated value of S_t and the better estimate $R_{t+1} + \gamma V(S_{t+1})$. This quantity, called the *TD error*, arises in various forms throughout RL:

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t) \tag{5.5}$$

5.2 Advantages of TD Prediction Methods

TD methods update their estimates based in part on other estimates. They learn a guess from a guess., i.e., they **bootstrap**.

TD methods have an advantage over DP methods in that they do not require a model of the environment, of its reward and next-state probability distributions. The next most obvious advantage of TD methods over Monte Carlo methods is that they are naturally implemented in an online, fully incremental fashion. With Monte Carlo methods one must wait until the end of an episode, because only then is the return known, whereas with TD methods one need wait only one time step.

Certainly it is convenient to learn one guess from the next, without waiting for an actual outcome, but can we still guarantee convergence to the correct answer? Happily, the answer is yes.

5.3 Optimality of TD(0)

Suppose that only a finite amount of experience is available. In this case, a common approach with incremental learning methods is to reuse the available experience repeatedly until the method converges to a stable solution.

Given an approximate value function (V), the increments specified by (5.1) or (5.2) are computed for every time step (t) at which a nonterminal state is visited. However, the value function is not updated after each individual increment. Instead, all increments are accumulated, and the value function is updated only once, using the sum of these increments.

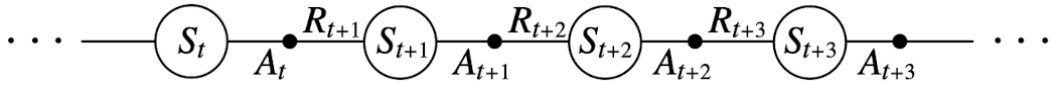
After this update, the entire batch of experience is processed again using the new value function, producing a new total increment. This procedure is repeated until the value function converges. This approach is called **batch updating**, because updates are performed only after processing the complete *batch* of training data.

Batch Monte Carlo methods converge to the estimates that minimize the mean squared error on the training set. In contrast, batch TD(0) converges to the estimates that would be exactly correct for the maximum-likelihood model of the Markov process. In general, the maximum-likelihood estimate of a parameter is the parameter value that assigns the highest probability to the observed data.

5.4 Sarsa: On-policy TD Control

We turn now to the use of TD prediction methods for the control problem. As with Monte Carlo methods, we face the need to trade off exploration and exploitation, and again approaches fall into two main classes: on-policy and off-policy. In this section we present an on-policy TD control method.

The first step is to learn an action-value function rather than a state-value function. We must estimate $q_\pi(S, a)$ for the current behavior policy π for all states s and actions a. This can be done using essentially the same TD method described above for learning v_π.



In the previous section we considered transitions from state to state and learned values of states. Now we consider transitions from state-action pair to state-action pair, and learn the values of state-action pairs. Formally these cases are identical: they are both Markov chains with a reward process. The theorems assuring the convergence of state values under TD(0) also apply to the corresponding algorith, for action values:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)\right]. \tag{5.6}$$

This rule uses every element of the quintuple of events, $(S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1})$, that make up a transition from one state-action pair to the next. This quintuple gives rise to the name **Sarsa** for the algorithm.

As in all on-policy methods, we continually estimate q_π for the behavior policy π, and at the same time change π toward greediness with respect to q_π.

Sarsa (on-policy TD control) for estimating $Q \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Choose A from S using policy derived from Q (e.g., ε-greedy)
  Loop for each step of episode:
    Take action A, observe R, S'
    Choose A' from S' using policy derived from Q (e.g., ε-greedy)
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma Q(S', A') - Q(S, A)\big]$
    $S \leftarrow S'$; $A \leftarrow A'$;
  until S is terminal

Figure 5.2: Pseudocode for Sarsa.

5.5 Q-learning: Off-policy TD Control

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{5.7}$$

This is an off-policy TD control algorithm known as **Q-learning**. Here, the learned action-value function, Q, directly approximates q_*, the optimal action-value function, independent of the policy being followed. The policy still has an effect in that it determines which state-action pairs are visited and updated. However, all that is required for the correct convergence is that all pairs continue to be updated. This is a minimal requirement in the sense that any method guaranteed to find optimal behavior in the general case must require it. Q has been shown to converge with probability 1 to q_*. The Q-learning algorithm is shown below:

Q-learning (off-policy TD control) for estimating $\pi \approx \pi_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, arbitrarily except that $Q(terminal, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using policy derived from Q (e.g., ε-greedy)
    Take action A, observe R, S'
    $Q(S, A) \leftarrow Q(S, A) + \alpha\big[R + \gamma \max_a Q(S', a) - Q(S, A)\big]$
    $S \leftarrow S'$
  until S is terminal

Figure 5.3: Pseudocode for Q-learning.

5.6 Expected Sarsa

Consider the learning algorithm that is just like Q-learning except that instead of the maximum over the next state-action pairs it uses the expected value, taking into account how likely each action is under the current policy. That is, consider the algorithm, with the update rule:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \mathbb{E}_\pi \left[Q(S_t, A_t) | S_{t+1}\right] - Q(S_t, A_t)\right] \tag{5.8}$$

$$= Q(S_t, A_t) + \alpha \left[R_{t+1} + \gamma \sum_a (a|S_{t+1}) Q(S_{t+1}, a) - Q(S_t, A_t)\right] \tag{5.9}$$

but that otherwise follows the schema of Q-learning. Given the next state, S_{t+1}, this algorithm moves *deterministically* in the same direction as Sarsa moves in *expectation*, and accordingly it is called **Expected Sarsa**.

Expected Sarsa is more complex computationally than Sarsa but, in return, it eliminates the variance due to the random selection of A_{t+1}.



Figure 5.4: The backup diagrams for Q-learning (left) and Expected Sarsa (right).

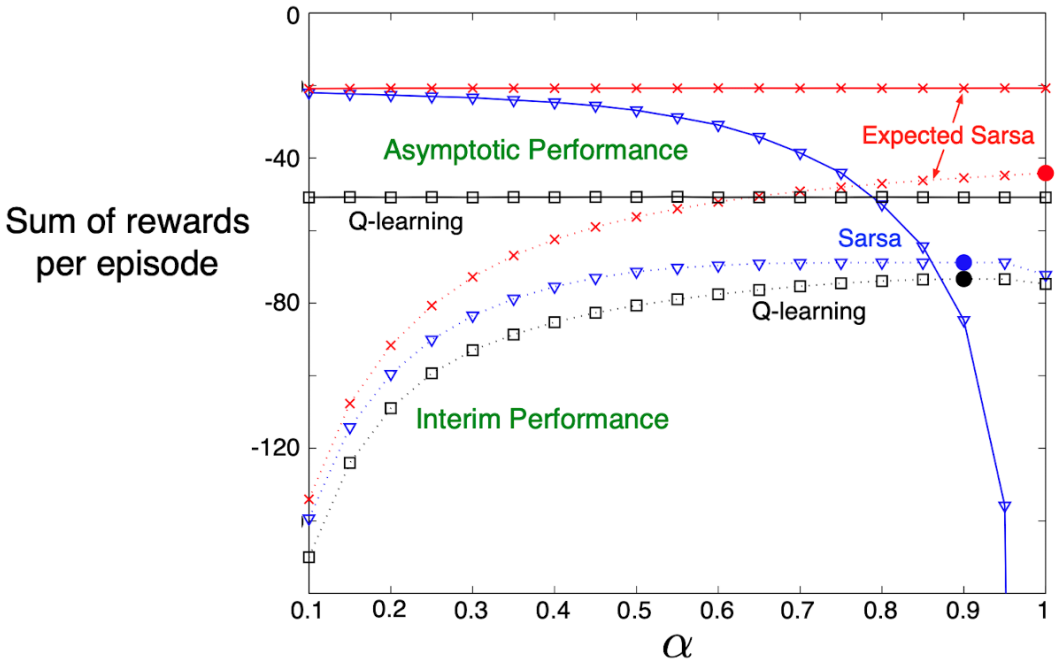


Figure 5.5: Interim and asymptotic performance of TD control methods on the cliff-walking task as a function of α. All algorithms used an ϵ-greedy policy with $\epsilon = 0.1$. Asymptotic performance is an average over 100,000 episodes whereas interim performance is an average over the first 100 episodes.

5.7 Maximization Bias and Double Learning

All the control algorithms that we have discussed so far involve a maximization in the construction of their target policies. In these algorithms, a maximum over estimated values is used implicitly as an estimate of the maximum value, which can lead to a significant positive bias. To see why, consider a single state s where there are many actions a whose true values, $q(s,a)$, are all zero but whose estimated values, $Q(s,a)$, are uncertain and thus distributed some above and some below zero. The maximum of the true values is zero, but the maximum of the estimates is positive, a positive bias. We call this *maximization bias*.

Are there algorithms that avoid this bias? Consider a bandit case in which we have noisy estimates of the value of each of many actions, obtained as simple averages of the rewards received on all the plays with each action. There will be a positive maximization bias if we use the maximum of the estimates as an estimate of the maximum of the true values.

Suppose we divided the plays in two sets and used them to learn two independent estimates, call them $Q_1(a)$ and $Q_2(a)$, each estimate of the true value $q(a)$, for all $a \in \mathcal{A}$. We could then use one estimate to determine the maximizing action $A^* = \arg\max_a Q_1(a)$ and the other, Q_2 in this case, to provide the estimate of its value, $Q_2(A^*) = Q_2(\arg\max_a Q_1(a))$. This estimate will then be unbiased in the sense that $\mathbb{E}\left[Q_2(A^*)\right] = q(A^*)$.

This is the idea of **double learning**. Note that although we learn two estimates, only one estimate is updated on each play; double learning doubles the memory requirements, but does not increase the amount of computation per step.

The idea of double learning extends naturally to algorithms for full MDPs. As an example, the double learning algorithm analogous to Q-learning, called Double Q-learning, divides the time steps in two, perhaps by flipping a coin on each step. If the coin comes up heads, the update is

$$Q_1(S_t,A_t) \leftarrow Q_1(S_t,A_t) + \alpha\left[R_{t+1} + \gamma Q_2(S_{t+1}, \arg\max_a Q_1(S_{t+1},a)) - Q_1(S_t,A_t)\right]. \tag{5.10}$$

If the coin comes up tails, then the same update is done with $Q-1$ and Q_2 switched, so that Q_2 is updated.

Double Q-learning, for estimating $Q_1 \approx Q_2 \approx q_*$

Algorithm parameters: step size $\alpha \in (0, 1]$, small $\varepsilon > 0$
Initialize $Q_1(s, a)$ and $Q_2(s, a)$, for all $s \in \mathcal{S}^+, a \in \mathcal{A}(s)$, such that $Q(\mathit{terminal}, \cdot) = 0$

Loop for each episode:
  Initialize S
  Loop for each step of episode:
    Choose A from S using the policy ε-greedy in $Q_1 + Q_2$
    Take action A, observe R, S'
    With 0.5 probabilility:
      $Q_1(S, A) \leftarrow Q_1(S, A) + \alpha\Big(R + \gamma Q_2\big(S', \operatorname{argmax}_a Q_1(S', a)\big) - Q_1(S, A)\Big)$
    else:
      $Q_2(S, A) \leftarrow Q_2(S, A) + \alpha\Big(R + \gamma Q_1\big(S', \operatorname{argmax}_a Q_2(S', a)\big) - Q_2(S, A)\Big)$
    $S \leftarrow S'$
  until S is terminal

Figure 5.6: The Double Learning algorithm.

Chapter 6

n-step Bootstrapping

Neither MC methods nor one-step TD methods are always the best. In this chapter we present **n-step TD methods** that generalize both approaches, so that one can shift from one to the other smoothly as needed to meet the demands of a particular task. n-step methods span a spectrum with MC methods at one end and one-step TD methods at the other. The best methods are often intermediate between the two extremes.

As usual, we first consider the prediction problem and then the control problem. That is, we first consider how n-step methods can help in predicting returns as a function of state for a fixed policy (i.e., in estimating v_π). Then we extend the ideas to action values and control methods.

6.1 n-step TD Prediction

Consider estimating v_π from sample episodes generated using π. Monte Carlo methods perform an update for each state based on the entire sequence of observed rewards from that state until the end of the episode. The update of one-step TD methods, on the other hand, is based on just the one next reward, bootstrapping from the value of the state one step later as a proxy for the remaining rewards. One kind of intermediate method, then, would perform an update based on an intermediate number of rewards.

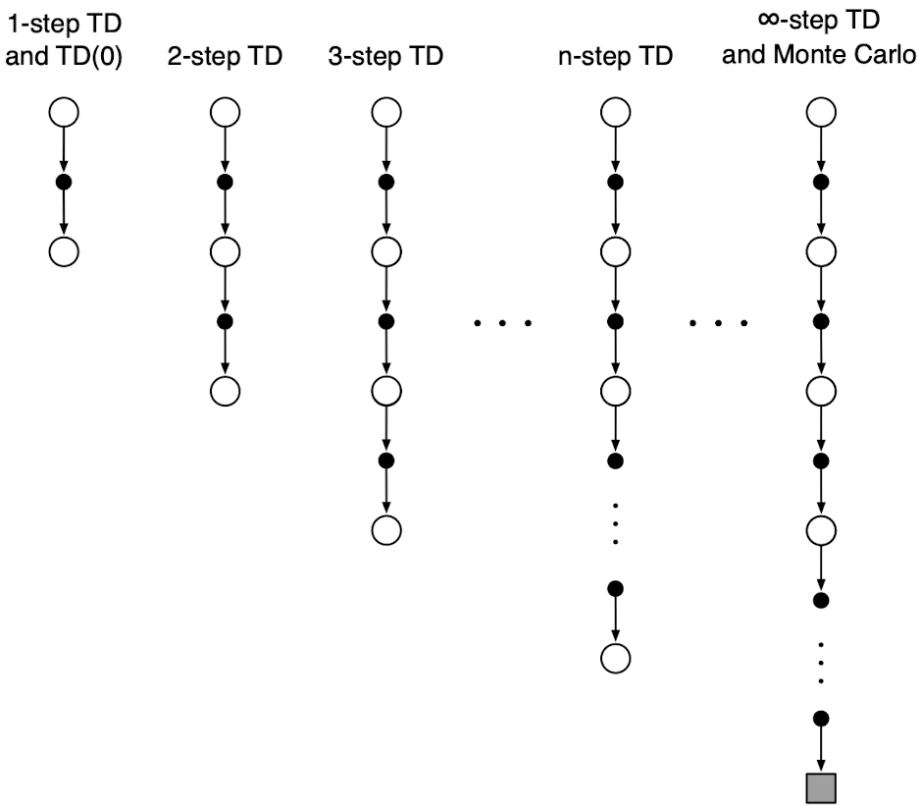


Figure 6.1: The backup diagram of n-step methods. These methods form a spectrum ranging from one-step TD methods to Monte Carlo methods.

Figure 6.1 shows the backup diagram of the spectrum of n-step updates for v_π, with the one-step TD update on the left and the up-until-termination Monte Carlo update on the right.

Methods in which the TD extends over n steps are called **n-step TD methods**. We know that in Monte Carlo updates the estimate of $v_\pi(S_t)$ is updated in the direction of the complete return:

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots + \gamma^{T-t-1} R_T \tag{6.1}$$

where T is the last time step of the episode. Let us call this quantity the *target* of the update. Whereas in Monte Carlo updates the target is the return, in one-step updates the target is the first reward plus the discounted estimated value of the next state, which we call the *one-step return*:

$$G_t^{t:t+1} = R_{t+1} + \gamma v_t(S_{t+1}) \tag{6.2}$$

where $V_t : \mathcal{S} \to \mathbb{R}$ here is the estimate at time t of v_π. Our point now is that this idea makes just as much sense after two steps as it does after one. The target for a two-step update is the *two-step return*:

$$G_t^{t:t+2} = R_{t+1} + \gamma R_{t+2} + \gamma^2 v_t(S_{t+2}) \tag{6.3}$$

Similarly, the target for an arbitrary n-step update is the *n-step return*:

$$G_t^{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n v_t(S_{t+n}) \tag{6.4}$$

The natural state-value learning algorithm for using n-step returns is thus:

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha \left[G_{t:t+n} - V_{t+n-1}(S_t) \right], \quad 0 \le t < T, \tag{6.5}$$

while the values of all other states remain unchanged: $V_{t+n}(S) = V_{t+n-1}(s)$, for all $s \neq S_t$. We call this algorithm n-step TD. Note that no changes at all are made during the first $n-1$ steps of each episode. To make up for that, an equal number of additional updates are made at the end of the episode, after termination and before starting the next episode.

n-step TD for estimating $V \approx v_\pi$

Input: a policy π
Algorithm parameters: step size $\alpha \in (0, 1]$, a positive integer n
Initialize $V(s)$ arbitrarily, for all $s \in \mathcal{S}$
All store and access operations (for S_t and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take an action according to $\pi(\cdot|S_t)$
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then $T \leftarrow t+1$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose state's estimate is being updated)
  | If $\tau \ge 0$:
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n V(S_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $V(S_\tau) \leftarrow V(S_\tau) + \alpha \left[G - V(S_\tau) \right]$
  Until $\tau = T - 1$

The n-step return uses the value function V_{t+n-1} to correct for the missing rewards beyond R_{t+n}. An important property of n-step returns is that their expectation is guaranteed to be a better estimate of v_π than V_{t+n-1} is, in a worst-state sense. That is, the worst error of the expected n-step return is guaranteed to be less than or equal to γ^n times the worst error under V_{t+n-1}:

$$\max_s \left| \mathbb{E}_\pi \left[G_{t:t+n} | S_t = s \right] - v_\pi(s) \right| \le \gamma^n \max_s \left| V_{t+n-1}(s) - v_\pi(s) \right|, \tag{6.6}$$

for all $n \ge 1$. This is called the **error reduction property of n-step returns**.

6.2 n-step Sarsa

How can n-step methods be used not just for prediction, but for control? The main idea is to simply switch states for actions (state-action pairs) and then use an ϵ-greedy policy. This is the n-step version of Sarsa and is called **n-step Sarsa**. We redefine n-step returns (update targets) in terms of estimated action values:

$$G_{t:t+n} = R_{t+1} + \gamma R_{t+2} + \cdots + \gamma^{n-1} R_{t+n} + \gamma^n Q(t+n-1)(S_{t+n}, A_{t+n}), n \geq 1, 0 \leq t < T-n, \tag{6.7}$$

with $G_{t:t+n} = G_t$ if $t+n \geq T$. The natural algorithm is then:

$$Q_{t+n}(S_t, A_t) = Q_{t+n-1}(S_t, A_t) + \alpha \left[G_{t:t+n} - Q_{t+n-1}(S_t, A_t) \right], \quad 0 \leq t < T, \tag{6.8}$$

while the values of all other states remain unchanged: $Q_{t+n}(s,a) = Q_{t+n-1}(s,a)$, for all s, a such that $s \neq S_t$ or $a \neq A_t$.

n-step Sarsa for estimating $Q \approx q_*$ or q_π

Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be ε-greedy with respect to Q, or to a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, small $\varepsilon > 0$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim \pi(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
    If $t < T$, then:
      Take action A_t
      Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
      If S_{t+1} is terminal, then:
        $T \leftarrow t+1$
      else:
        Select and store an action $A_{t+1} \sim \pi(\cdot|S_{t+1})$
    $\tau \leftarrow t-n+1$ (τ is the time whose estimate is being updated)
    If $\tau \geq 0$:
      $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n,T)} \gamma^{i-\tau-1} R_i$
      If $\tau + n < T$, then $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
      $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha \left[G - Q(S_\tau, A_\tau) \right]$
      If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is ε-greedy wrt Q
  Until $\tau = T-1$

Figure 6.2: The n-step Sarsa algorithm. The n-step return is used to update the action value function. The policy is ϵ-greedy with respect to the action value function.

6.3 n-step Off-policy Learning

In n-step methods, returns are constructed over n steps, so we are interested in the relative probability of just those n actions. To make a simple off-policy version of n-step TD, the update for time t (actually made at time $t+n$) can simply be weighted by $\rho_{t:t+n-1}$:

$$V_{t+n}(S_t) = V_{t+n-1}(S_t) + \alpha \rho_{t:t+n-1} \left[G_{t:t+n} - V_{t+n-1}(S_t) \right], \quad 0 \leq t < T, \tag{6.9}$$

where $\rho_{t:t+n-1}$, called the *importance sampling ratio*, is the relative probability under the two policies of taking the n actions from A_t to A_{t+n-1}:

$$\rho_{t:h} = \prod_{k=t}^{\min(h,T-1)} \frac{\pi(A_k|S_k)}{b(A_k|S_k)} \tag{6.10}$$

Off-policy n-step Sarsa for estimating $Q \approx q_*$ or q_π

Input: an arbitrary behavior policy b such that $b(a|s) > 0$, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize $Q(s,a)$ arbitrarily, for all $s \in \mathcal{S}, a \in \mathcal{A}$
Initialize π to be greedy with respect to Q, or as a fixed given policy
Algorithm parameters: step size $\alpha \in (0,1]$, a positive integer n
All store and access operations (for S_t, A_t, and R_t) can take their index mod $n+1$

Loop for each episode:
  Initialize and store $S_0 \neq$ terminal
  Select and store an action $A_0 \sim b(\cdot|S_0)$
  $T \leftarrow \infty$
  Loop for $t = 0, 1, 2, \ldots$:
  | If $t < T$, then:
  | Take action A_t
  | Observe and store the next reward as R_{t+1} and the next state as S_{t+1}
  | If S_{t+1} is terminal, then:
  | $T \leftarrow t+1$
  | else:
  | Select and store an action $A_{t+1} \sim b(\cdot|S_{t+1})$
  | $\tau \leftarrow t - n + 1$ (τ is the time whose estimate is being updated)
  | If $\tau \geq 0$:
  | $\rho \leftarrow \prod_{i=\tau+1}^{\min(\tau+n, T-1)} \frac{\pi(A_i|S_i)}{b(A_i|S_i)}$ $(\rho_{\tau+1:\tau+n})$
  | $G \leftarrow \sum_{i=\tau+1}^{\min(\tau+n, T)} \gamma^{i-\tau-1} R_i$
  | If $\tau + n < T$, then: $G \leftarrow G + \gamma^n Q(S_{\tau+n}, A_{\tau+n})$ $(G_{\tau:\tau+n})$
  | $Q(S_\tau, A_\tau) \leftarrow Q(S_\tau, A_\tau) + \alpha\rho\left[G - Q(S_\tau, A_\tau)\right]$
  | If π is being learned, then ensure that $\pi(\cdot|S_\tau)$ is greedy wrt Q
  Until $\tau = T - 1$

Figure 6.3: The n-step off-policy learning algorithm. The importance sampling ratio $\rho_{t:t+n-1}$ is used to weight the update. The policy is ϵ-greedy with respect to the action value function.

Chapter 7

Planning and Learning with Tabular Methods

In this chapter we develop a unified view of reinforcement learning methods that require a model of the environment, such as dynamic programming and heuristic search, and methods that can be used without a model, such as Monte Carlo and temporal-difference methods. These are respectively called **model-based** and **model-free** reinforcement learning methods. Model-based methods rely on planning as their primary component, while model-free methods primarily rely on learning. Although there are real differences between these two kinds of methods, there are also great similarities. In particular, **the heart of both kinds of methods is the computation of value functions**. Moreover, all the methods are based on looking ahead to future events, computing a backed-up value, and then using it as an update target for an approximate value function.

7.1 Models and Planning

By a model of the environment we mean anything that an agent can use to predict how the environment will respond to its actions. Given a state and an action, a model produces a prediction of the resultant next state and next reward. If the model is stochastic, then there are several possible next states and next rewards, each with some probability of occurring. Some models produce a description of all possibilities and their probabilities; these we call **distribution models**. Other models produce just one of the possibilities, sampled according to the probabilities; these we call **sample models**.

The kind of model assumed in dynamic programming—estimates of the MDP's dynamics, $p(s', r|s, a)$—is a distribution model. The kind of model used in the blackjack example in Chapter 5 is a sample model. Distribution models are stronger than sample models in that they can always be used to produce samples. However, in many applications it is much easier to obtain sample models than distribution models.

Models can be used to mimic or simulate experience. Given a starting state and action, a sample model produces a possible transition, and a distribution model generates all possible transitions weighted by their probabilities of occurring. Given a starting state and a policy, a sample model could produce an entire episode, and a distribution model could generate all possible episodes and their probabilities. In either case, we say the model is used to simulate the environment and produce simulated experience.

The word planning is used in several different ways in different fields. We use the term to refer to any computational process that takes a model as input and produces or improves a policy for interacting with the modeled environment:

$$\text{model} \rightarrow \text{planning} \rightarrow \text{policy}$$

State-space planning, which includes the approach we take in this book, is viewed primarily as a search through the state space for an optimal policy or an optimal path to a goal. Actions cause transitions from state to state, and value functions are computed over states. In what we call plan-space planning, planning is instead a search through the space of plans. Operators transform one plan into another, and value functions, if any, are defined over the space of plans.

The unified view we present in this chapter is that all state-space planning methods share a common structure, a structure that is also present in the learning methods presented in this book. It takes the rest of the chapter to develop this view, but there are two basic ideas: (1) all state-space planning methods involve computing value functions as a key intermediate step toward improving the policy, and (2) they compute value functions by updates or backup operations applied to simulated experience.

Dynamic programming methods clearly fit this structure: they make sweeps through the space of states, generating for each state the distribution of possible transitions. Each distribution is then used to compute a backed-up value

(update target) and update the state's estimated value. Viewing planning methods in this way emphasizes their relationship to the learning methods that we have described in this book. The heart of both learning and planning methods is the estimation of value functions by backing-up update operations. The difference is that whereas planning uses simulated experience generated by a model, learning methods use real experience generated by the environment.

Random-sample one-step tabular Q-planning

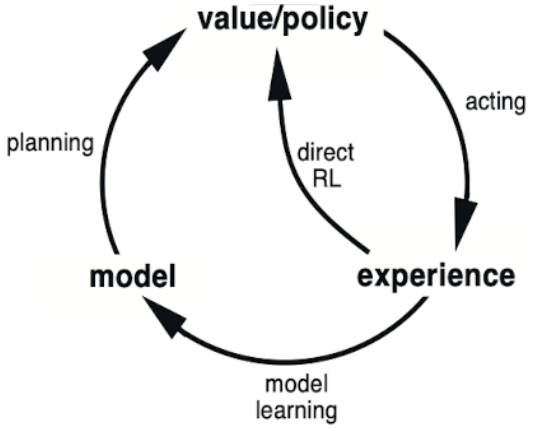
Loop forever:

1. Select a state, $S \in \mathcal{S}$, and an action, $A \in \mathcal{A}(S)$, at random
2. Send S, A to a sample model, and obtain
   a sample next reward, R, and a sample next state, S'
3. Apply one-step tabular Q-learning to S, A, R, S':
   $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

7.2 Dyna: Integrated Planning, Acting, and Learning

When planning is done online, while interacting with the environment, a number of interesting issues arise. New information gained from the interaction may change the model and thereby interact with planning. It may be desirable to customize the planning process in some way to the states or decisions currently under consideration, or expected in the near future. If decision making and model learning are both computation-intensive processes, then the available computational resources may need to be divided between them. To begin exploring these issues, in this section we present Dyna-Q, a simple architecture integrating the major functions needed in an online planning agent.

Within a planning agent, there are at least two roles for real experience: it can be used to improve the model (to make it more accurately match the real environment) and it can be used to directly improve the value function and policy using the kinds of reinforcement learning methods we have discussed in previous chapters. The former we call **model-learning**, and the latter we call **direct reinforcement learning** (direct RL).



Both direct and indirect methods have advantages and disadvantages. Indirect methods often make fuller use of a limited amount of experience and thus achieve a better policy with fewer environmental interactions. On the other hand, direct methods are much simpler and are not affected by biases in the design of the model. Some have argued that indirect methods are always superior to direct ones, while others have argued that direct methods are responsible for most human and animal learning.

Dyna-Q includes all of the processes shown in the diagram above—planning, acting, model-learning, and direct RL—all occurring continually. The planning method is the random-sample one-step tabular Q-planning method. The direct RL method is one-step tabular Q-learning. The model-learning method is also table-based and assumes the environment is deterministic. After each transition $S_t, A_t \to R_{t+1}, S_{t+1}$, the model records in its table entry for S_t, A_t the prediction that R_{t+1}, S_{t+1} will deterministically follow. Thus, if the model is queried with a state–action pair that has been experienced before, it simply returns the last-observed next state and next reward as its prediction.

During planning, the Q-planning algorithm randomly samples only from state–action pairs that have previously been experienced (in Step 1), so the model is never queried with a pair about which it has no information.

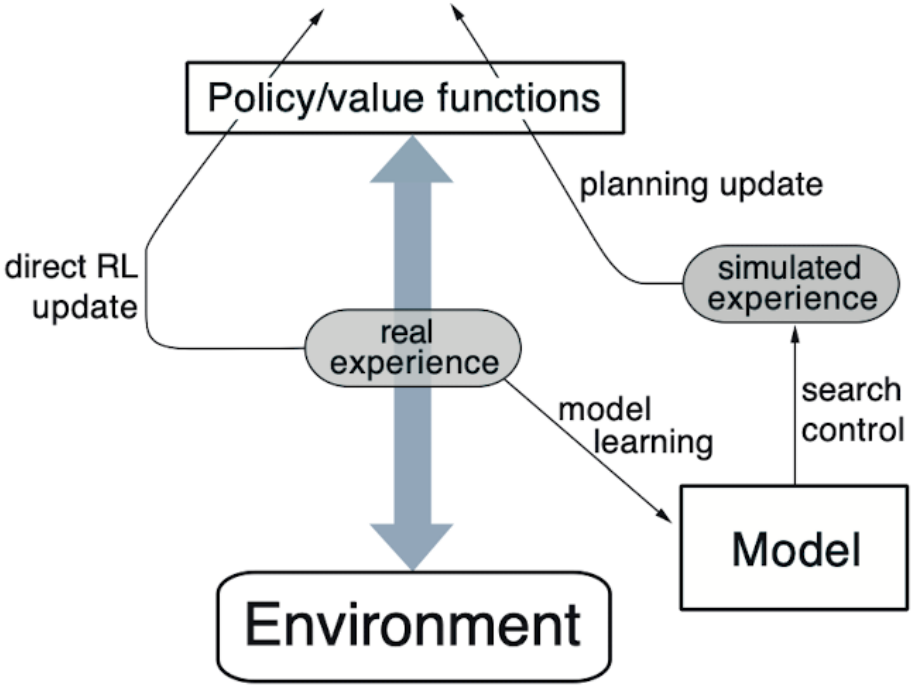


Figure 7.1: The general Dyna Architecture.

Conceptually, planning, acting, model-learning, and direct RL occur simultaneously and in parallel in Dyna agents. For concreteness and implementation on a serial computer, however, we fully specify the order in which they occur within a time step. In Dyna-Q, the acting, model-learning, and direct RL processes require little computation, and we assume they consume just a fraction of the time. The remaining time in each step can be devoted to the planning process, which is inherently computation-intensive.

Tabular Dyna-Q

Initialize $Q(s,a)$ and $Model(s,a)$ for all $s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow \varepsilon$-greedy(S,Q)
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
- (e) $Model(S,A) \leftarrow R, S'$ (assuming deterministic environment)
- (f) Loop repeat n times:
  - $S \leftarrow$ random previously observed state
  - $A \leftarrow$ random action previously taken in S
  - $R, S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$

7.3 When the Model Is Wrong

In the maze example presented in the previous section, the changes in the model were relatively modest. The model started out empty, and was then filled only with exactly correct information. In general, we cannot expect to be so fortunate. Models may be incorrect because the environment is stochastic and only a limited number of samples have been observed, or because the model was learned using function approximation that has generalized imperfectly, or simply because the environment has changed and its new behavior has not yet been observed. When the model is incorrect, the planning process is likely to compute a suboptimal policy.

In some cases, the suboptimal policy computed by planning quickly leads to the discovery and correction of the modeling error. This tends to happen when the model is optimistic in the sense of predicting greater reward or better state transitions than are actually possible. The planned policy attempts to exploit these opportunities and in doing so discovers that they do not exist.

Greater difficulties arise when the environment changes to become better than it was before, and yet the formerly correct policy does not reveal the improvement. In these cases the modeling error may not be detected for a long

time, if ever.

The general problem here is another version of the conflict between exploration and exploitation. In a planning context, exploration means trying actions that improve the model, whereas exploitation means behaving in the optimal way given the current model.

We want the agent to explore to find changes in the environment, but not so much that performance is greatly degraded. As in the earlier exploration/exploitation conflict, there probably is no solution that is both perfect and practical, but simple heuristics are often effective.

7.4 Prioritized Sweeping

In the Dyna agents presented in the preceding sections, simulated transitions are started in state–action pairs selected uniformly at random from all previously experienced pairs. But a uniform selection is usually not the best; planning can be much more efficient if simulated transitions and updates are focused on particular state–action pairs.

This example suggests that search might be usefully focused by working backward from goal states. Of course, we do not really want to use any methods specific to the idea of "goal state." We want methods that work for general reward functions. Goal states are just a special case, convenient for stimulating intuition. In general, we want to work back not just from goal states but from any state whose value has changed. Suppose that the values are initially correct given the model. Suppose now that the agent discovers a change in the environment and changes its estimated value of one state, either up or down. Typically, this will imply that the values of many other states should also be changed, but the only useful one-step updates are those of actions that lead directly into the one state whose value has been changed.

This general idea might be termed **backward focusing of planning computations**.

It is natural to prioritize the updates according to a measure of their urgency, and perform them in order of priority. This is the idea behind **prioritized sweeping**. A queue is maintained of every state–action pair whose estimated value would change nontrivially if updated, prioritized by the size of the change. When the top pair in the queue is updated, the effect on each of its predecessor pairs is computed. If the effect is greater than some small threshold, then the pair is inserted in the queue with the new priority. In this way the effects of changes are efficiently propagated backward until quiescence.

Prioritized sweeping is just one way of distributing computations to improve planning efficiency, and probably not the best way. One of prioritized sweeping's limitations is that it uses expected updates, which in stochastic environments may waste lots of computation on low-probability transitions.

Prioritized sweeping for a deterministic environment

Initialize $Q(s,a)$, $Model(s,a)$, for all s,a, and $PQueue$ to empty
Loop forever:
- (a) $S \leftarrow$ current (nonterminal) state
- (b) $A \leftarrow policy(S,Q)$
- (c) Take action A; observe resultant reward, R, and state, S'
- (d) $Model(S,A) \leftarrow R, S'$
- (e) $P \leftarrow |R + \gamma \max_a Q(S',a) - Q(S,A)|$.
- (f) if $P > \theta$, then insert S,A into $PQueue$ with priority P
- (g) Loop repeat n times, while $PQueue$ is not empty:
  - $S,A \leftarrow first(PQueue)$
  - $R,S' \leftarrow Model(S,A)$
  - $Q(S,A) \leftarrow Q(S,A) + \alpha\big[R + \gamma \max_a Q(S',a) - Q(S,A)\big]$
  - Loop for all $\bar{S},\bar{A}$ predicted to lead to S:
    - $\bar{R} \leftarrow$ predicted reward for $\bar{S},\bar{A},S$
    - $P \leftarrow |\bar{R} + \gamma \max_a Q(S,a) - Q(\bar{S},\bar{A})|$.
    - if $P > \theta$ then insert $\bar{S},\bar{A}$ into $PQueue$ with priority P

We have suggested in this chapter that all kinds of state-space planning can be viewed as sequences of value updates, varying only in the type of update, expected or sample, large or small, and in the order in which the updates are done.

7.5 Expected vs. Sample Updates

Much of this book has been about different kinds of value-function updates, and we have considered a great many varieties. Focusing for the moment on one-step updates, they vary primarily along three binary dimensions. The first two dimensions are whether they update state values or action values and whether they estimate the value for the optimal policy or for an arbitrary given policy. These two dimensions give rise to four classes of updates for approximating the four value functions, q_*, v_*, q_π, and v_π. The other binary dimension is whether the updates are **expected updates**, considering all possible events that might happen, or **sample updates**, considering a single sample of what might happen.

When we introduced one-step sample updates in Chapter 6, we presented them as substitutes for expected updates. In the absence of a distribution model, expected updates are not possible, but sample updates can be done using sample transitions from the environment or a sample model. Implicit in that point of view is that expected updates, if possible, are preferable to sample updates. But are they? Expected updates certainly yield a better estimate because they are uncorrupted by sampling error, but they also require more computation, and computation is often the limiting resource in planning.

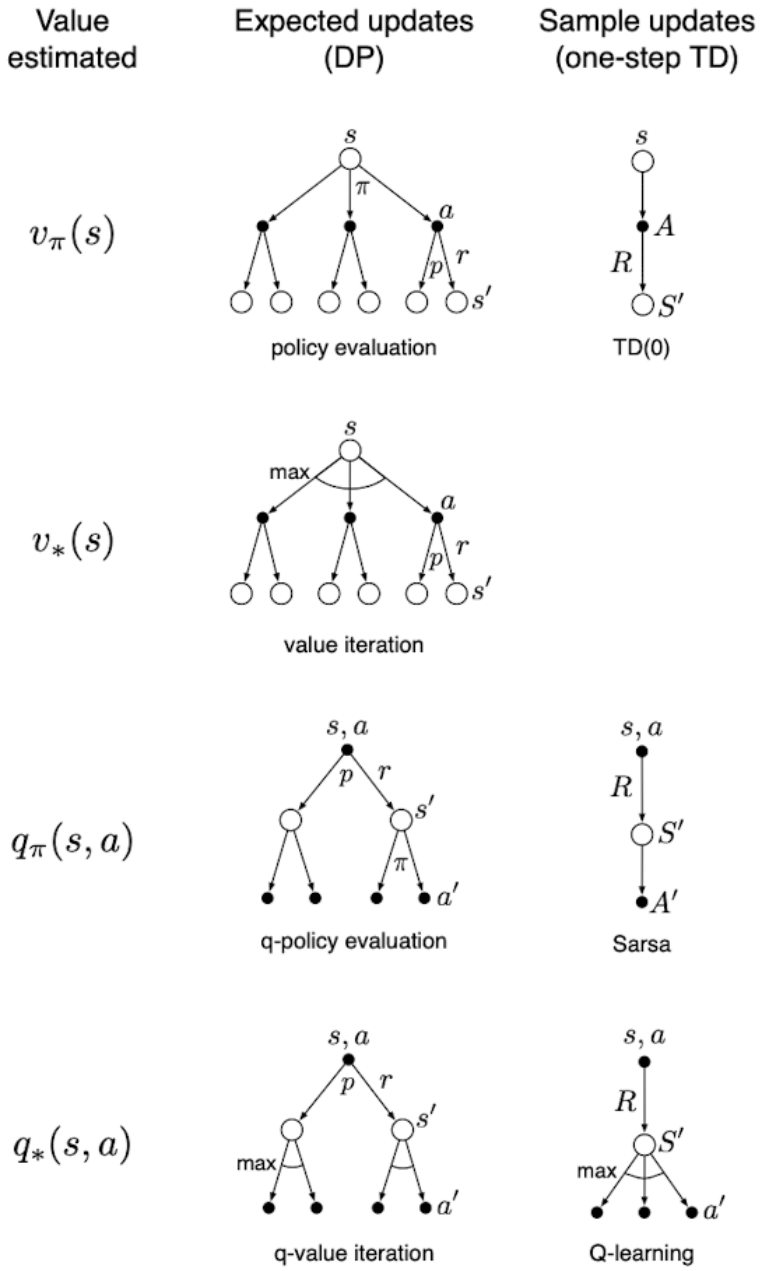


Figure 7.2: Backup diagrams for all the one-step updates considered in this chapter.

For concreteness, consider the expected and sample updates for approximating q_*, and the special case of discrete states and actions, a table-lookup representation of the approximate value function, Q, and a model in the form

of estimated dynamics, $\hat{p}(s', r|s, a)$. The expected update for a state–action pair, s, a, is:

$$Q(s, a) \leftarrow \sum_{s', r} \hat{p}(s', r|s, a) \left[r + \gamma \max_{a'} Q(s', a') \right] \tag{7.1}$$

The corresponding sample update for s, a, given a sample next state and reward, S' and R (from the model), is the Q-learning-like update:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[R + \gamma \max_{a'} Q(S', a') - Q(s, a) \right] \tag{7.2}$$

If there is enough time to complete an expected update, then the resulting estimate is generally better than that of b sample updates because of the absence of sampling error. But if there is insufficient time to complete an expected update, then sample updates are always preferable because they at least make some improvement in the value estimate with fewer than b updates.

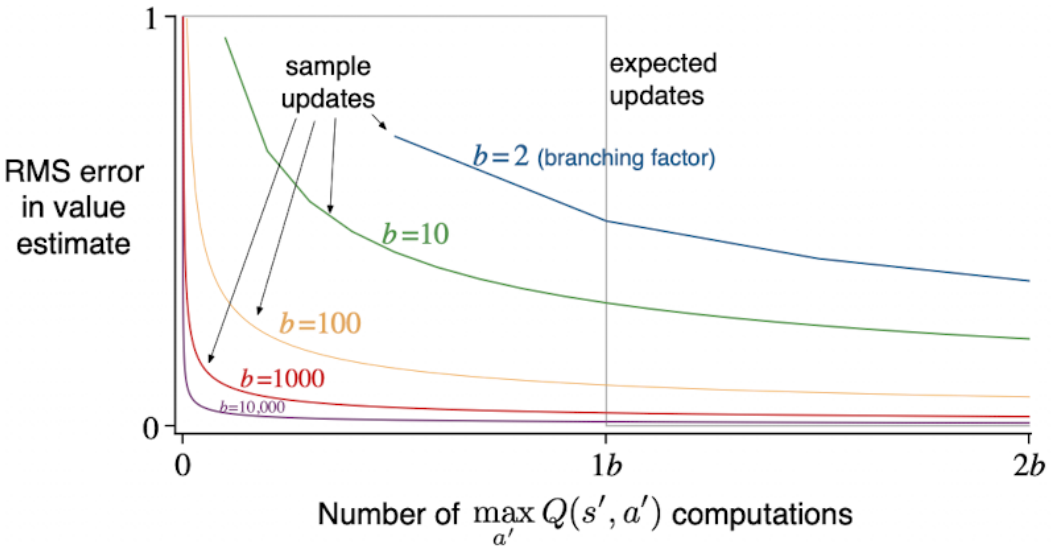


Figure 7.3: Comparison of efficiency of expected and sample updates.

7.6 Trajectory Sampling

In this section we compare two ways of distributing updates. The classical approach, from dynamic programming, is to perform sweeps through the entire state (or state–action) space, updating each state (or state–action pair) once per sweep. This is problematic on large tasks because there may not be time to complete even one sweep. In many tasks the vast majority of the states are irrelevant because they are visited only under very poor policies or with very low probability.

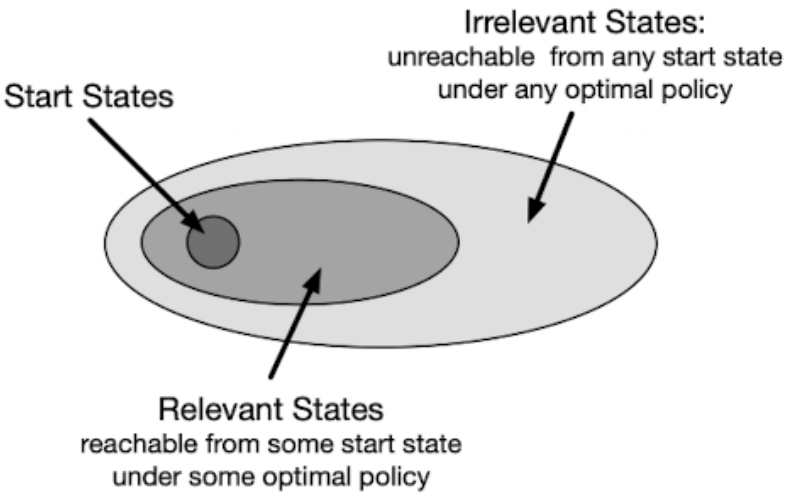
The second approach is to sample from the state or state–action space according to some distribution. One could sample uniformly, as in the Dyna-Q agent, but this would suffer from some of the same problems as exhaustive sweeps. More appealing is to distribute updates according to the **on-policy distribution**, that is, according to the distribution observed when following the current policy. One advantage of this distribution is that it is easily generated; one simply interacts with the model, following the current policy. We call this way of generating experience and updates **trajectory sampling**.

Focusing on the on-policy distribution could be beneficial because it causes vast, uninteresting parts of the space to be ignored, or it could be detrimental because it causes the same old parts of the space to be updated over and over.

7.7 Real-time Dynamic Programming

Real-time dynamic programming, or RTDP, is an on-policy trajectory-sampling version of the value-iteration algorithm of dynamic programming (DP). Because it is closely related to conventional sweep-based policy iteration, RTDP illustrates in a particularly clear way some of the advantages that on-policy trajectory sampling can provide. RTDP updates the values of states visited in actual or simulated trajectories by means of expected tabular value-iteration updates.

If trajectories can start only from a designated set of start states, and if you are interested in the prediction problem for a given policy, then on-policy trajectory sampling allows the algorithm to completely skip states that cannot be reached by the given policy from any of the start states: such states are irrelevant to the prediction problem. For a control problem, where the goal is to find an optimal policy instead of evaluating a given policy, there might well be states that cannot be reached by any optimal policy from any of the start states, and there is no need to specify optimal actions for these irrelevant states. What is needed is an **optimal partial policy**, meaning a policy that is optimal for the relevant states but can specify arbitrary actions, or even be undefined, for the irrelevant states.



The most interesting result for RTDP is that for certain types of problems satisfying reasonable conditions, RTDP is guaranteed to find a policy that is optimal on the relevant states without visiting every state infinitely often, or even without visiting some states at all. Indeed, in some problems, only a small fraction of the states need to be visited. This can be a great advantage for problems with very large state sets, where even a single sweep may not be feasible.

7.8 Planning at Decision Time

Planning can be used in at least two ways. The one we have considered so far in this chapter, typified by dynamic programming and Dyna, is to use planning to gradually improve a policy or value function on the basis of simulated experience obtained from a model. We call planning used in this way **background planning**.

The other way to use planning is to begin and complete it after encountering each new state S_t, as a computation whose output is the selection of a single action A_t; on the next step planning begins anew with S_{t+1} to produce A_{t+1}, and so on. Unlike the first use of planning, here planning focuses on a particular state. We call this **decision-time planning**.

7.9 Heuristic Search

The classical state-space planning methods in artificial intelligence are decision-time planning methods collectively known as **heuristic search**. In heuristic search, for each state encountered, a large tree of possible continuations is considered. The approximate value function is applied to the leaf nodes and then backed up toward the current state at the root. The backing up within the search tree is just the same as in the expected updates with maxes discussed throughout this book. The backing up stops at the state–action nodes for the current state. Once the backed-up values of these nodes are computed, the best of them is chosen as the current action, and then all backed-up values are discarded.

In conventional heuristic search no effort is made to save the backed-up values by changing the approximate value function. In fact, the value function is generally designed by people and never changed as a result of search.

We should not overlook the most obvious way in which heuristic search focuses updates: on the current state. Much of the effectiveness of heuristic search is due to its search tree being tightly focused on the states and actions that might immediately follow the current state.

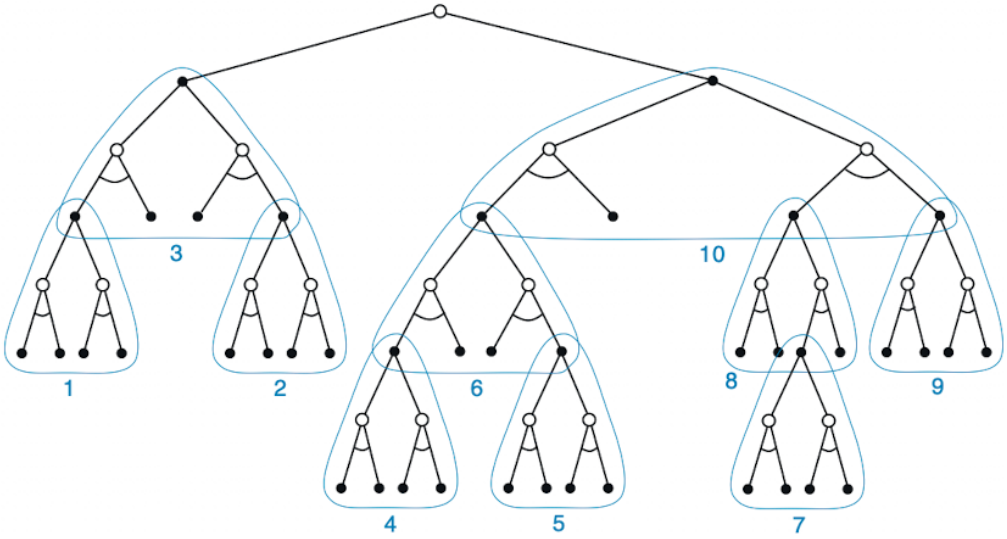


Figure 7.4: Heuristic search can be implemented as a sequence of one-step updates (blu lines) backing up values from the leaf nodes toward the root. The ordering shown here is for a selective depth-first search.

7.10 Rollout Algorithms

Rollout algorithms are decision-time planning algorithms based on Monte Carlo control applied to simulated trajectories that all begin at the current environment state. They estimate action values for a given policy by averaging the returns of many simulated trajectories that start with each possible action and then follow the given policy. When the action-value estimates are considered to be accurate enough, the action (or one of the actions) having the highest estimated value is executed, after which the process is carried out anew from the resulting next state.

What then do rollout algorithms accomplish? The policy improvement theorem tells us that given any two policies π and π' that are identical except that $\pi'(s) = a \neq \pi(s)$ for some state s, if $q_\pi(s, a) \geq v_\pi(s)$, then policy π' is as good as, or better, than π. This applies to rollout algorithms where s is the current state and π is the rollout policy.

In other words, the aim of a rollout algorithm is to improve upon the rollout policy; not to find an optimal policy. Experience has shown that rollout algorithms can be surprisingly effective.

7.11 Monte Carlo Tree Search

Monte Carlo Tree Search (MCTS) is a recent and strikingly successful example of decision-time planning. At its base, MCTS is a rollout algorithm as described above, but enhanced by the addition of a means for accumulating value estimates obtained from the Monte Carlo simulations in order to successively direct simulations toward more highly-rewarding trajectories. MCTS is largely responsible for the improvement in computer Go from a weak amateur level in 2005 to a grandmaster level (6 dan or more) in 2015.

MCTS is executed after encountering each new state to select the agent's action for that state; it is executed again to select the action for the next state, and so on. The core idea of MCTS is to successively focus multiple simulations starting at the current state by extending the initial portions of trajectories that have received high evaluations from earlier simulations.

For the most part, the actions in the simulated trajectories are generated using a simple policy, usually called a rollout policy as it is for simpler rollout algorithms. Monte Carlo value estimates are maintained only for the subset of state–action pairs that are most likely to be reached in a few steps, which form a tree rooted at the current state. MCTS incrementally extends the tree by adding nodes representing states that look promising based on the results of the simulated trajectories.

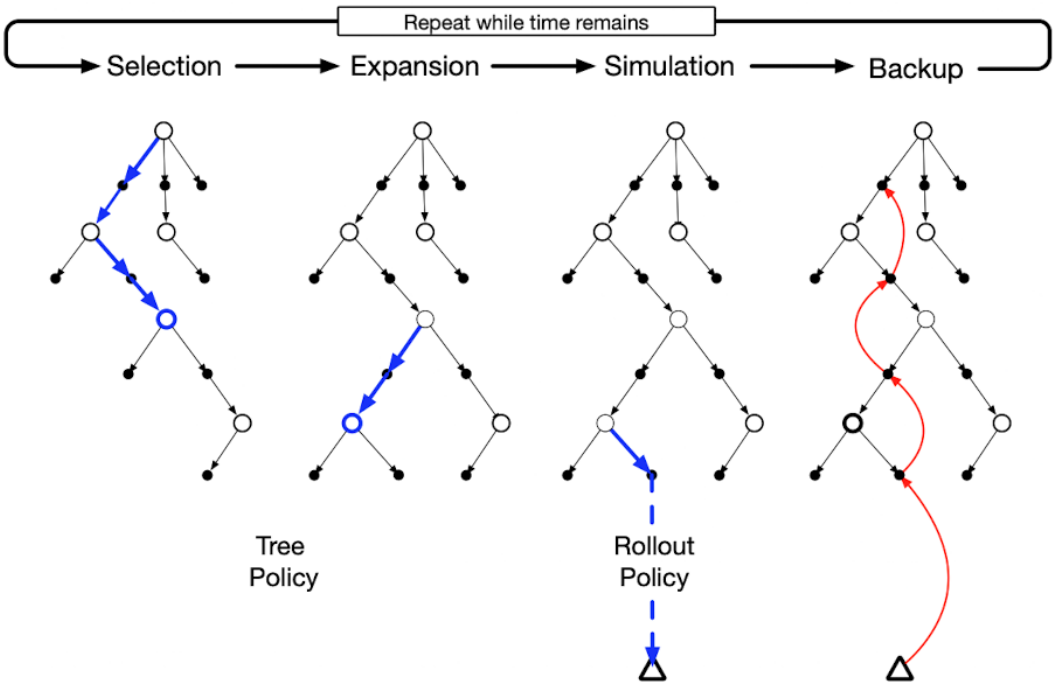


Figure 7.5: Monte Carlo Tree Search. When the environment changes to a new state, MCTS executes as many iterations as possible before an action needs to be selected, incrementally building a tree whose root node represents the current state. Each iteration consists of the four operations **Selection**, **Expansion**, **Simulation** and **Backup**.

Each iteration of a basica version of MCTS consists of the following four steps:

1. **Selection.** Starting at the root node, a tree policy base on the action values attached to the edges of the tree traverses the tree to select a leaf node.
2. **Expansion.** On some iterations, the tree is expanded from the selected leaf node by adding one or ore child nodes reached from the selected node via unexplored actions.
3. **Simulation.** From the selected node or from one of its newly-added child nodes, simulation of a complete episode is run with actions selected by the rolloud policy. The result is a MC trial with actions selected first by the tree policy and beyond the tree by the rollout policy.
4. **Backup.** The return generated by the simulated episode is backed up to update or to initialize, the action values attached to the edges of the tree traversed by the tree policy in this iteration of MCTS. No value are saved for the states and actions visited by the rollout policy beyond the tree.

Chapter 8

Approximate Solution Methods

The novelty in this chapter is that the approximate value function is represented not as a table but as a parameterized functional form with weight vector $\mathbf{w} \in \mathbb{R}$. We will write $\hat{v}(s, w) \approx v_\pi(s)$ for the approximate value of state s given weight vector $\mathbf{w}$.

Typically, the number of weights is much less than the number of states, consequently, when a single state is updated, the change generalizes from that state to affect the values of many other states. Such generalization makes the learning potentially more powerful byt also potentially more difficult to manage and understand. Extending RL to function approximation also makes it applicable to partially observable problems, in which the full state is not available to the agent.

What function approximation can't do, however, is augment the state representation with memories of past observations.

8.1 Value-function Approximation

All of the prediction methods covered in this book have been described as updates to an estimated value function that shifts its value at particular states toward a "backed-up value", or *update target*, for that state. Let us refer to an individual update by the notation $s \to u$, where s is the state updated and u is the update target that s's estimated value is shifted toward.

It is natural to interpret each update as specifying an example of the desired input-output behavior of the value function. In a sense, the update $s \to u$ means that the estimated value for state s should be more like the update target u. Up to now, the actual update has been trivial: the table entry for s's estimated value has simply been shifted a fraction of the way toward u, and the estimated values of all other states were left unchanged. Now we permit arbitrarily complex and sophisticated methods to implement the update, and updating at s generalizer so that the estimated values of many other states are changed as well.

Function approximation methods expect to receive examples of the desired input-output behavior of the function they are trying to approximate. We use these methods for value prediction simply by passing to them the $s \to u$ of each update as a training example. We then interpret the approximate function they produce as an estimated value function. In RL, however, it is imoirtant that learning be able to occur online, while the agent interacts with its environment or with a model of its environment. To do this requires methods that are able to learn efficiently from incrementally acquired data. In addition, RL generally requires function approximation methods able to handle nonstationary target functions.

8.2 The Prediction Objective ($\overline{VE}$)

Up to now we have not specified an explicit objective for prediction. In the tabular case a continuous measure of prediction quality was not necessary because the learned value function could come to equal the true value function exactly. Moreover, the learned values at each state were decoupled—an update at one state affected no other. But with genuine approximation, an update at one state affects many others, and it is not possible to get the values of all states exactly correct. By assumption we have far more states than weights, so making one state's estimate more accurate invariably means making others' less accurate. We are obligated then to say which states we care most about. We must specify a state distribution $\mu(s) \geq 0$, $\sum_s \mu(s) = 1$, representing how much we care about the error in each state s. By the error in a state s we mean the square of the difference between the approximate value $\hat{v}(s, w)$ and the true value $v_\pi(s)$. Weighting this over the state space by μ, we obtain a natural objective function, the mean square value error, denoted $\overline{VE}$:

$$\overline{VE}(w) \doteq \sum_{s \in S} \mu(s)[v_\pi(s) - \hat{v}(s,w)]^2$$

The square root of this measure, the root $\overline{VE}$, gives a rough measure of how much the approximate values differ from the true values and is often used in plots. Often $\mu(s)$ is chosen to be the fraction of time spent in s. Under on-policy training this is called the on-policy distribution; we focus entirely on this case in this chapter. In continuing tasks, the on-policy distribution is the stationary distribution under π.

8.3 Stochastic-gradient and Semi-gradient Methods

We now develop in detail one class of learning methods for function approximation in value prediction, those based on stochastic gradient descent (SGD). SGD methods are among the most widely used of all function approximation methods and are particularly well suited to online reinforcement learning.

In gradient-descent methods, the weight vector is a column vector with a fixed number of real valued components, $w \doteq (w_1, w_2, \ldots, w_d)^T$, and the approximate value function $\hat{v}(s,w)$ is a differentiable function of w for all $s \in S$. We will be updating w at each of a series of discrete time steps, $t = 0, 1, 2, 3, \ldots$, so we will need a notation w_t for the weight vector at each step. For now, let us assume that, on each step, we observe a new example $S_t \to v_\pi(S_t)$ consisting of a (possibly randomly selected) state S_t and its true value under the policy. These states might be successive states from an interaction with the environment, but for now we do not assume so. Even though we are given the exact, correct values, $v_\pi(S_t)$ for each S_t, there is still a difficult problem because our function approximator has limited resources and thus limited resolution. In particular, there is generally no w that gets all the states, or even all the examples, exactly correct. In addition, we must generalize to all the other states that have not appeared in examples.

We assume that states appear in examples with the same distribution, μ, over which we are trying to minimize the $\overline{VE}$ as given by (9.1). A good strategy in this case is to try to minimize error on the observed examples. Stochastic gradient-descent (SGD) methods do this by adjusting the weight vector after each example by a small amount in the direction that would most reduce the error on that example:

$$w_{t+1} \doteq w_t - \frac{1}{2}\alpha \nabla [v_\pi(S_t) - \hat{v}(S_t, w_t)]^2$$

$$= w_t + \alpha [v_\pi(S_t) - \hat{v}(S_t, w_t)] \nabla \hat{v}(S_t, w_t)$$

where α is a positive step-size parameter, and $\nabla f(w)$, for any scalar expression $f(w)$ that is a function of a vector (here w), denotes the column vector of partial derivatives of the expression with respect to the components of the vector:

$$\nabla f(w) \doteq \left(\frac{\partial f(w)}{\partial w_1}, \frac{\partial f(w)}{\partial w_2}, \ldots, \frac{\partial f(w)}{\partial w_d} \right)^T$$

This derivative vector is the gradient of f with respect to w. SGD methods are "gradient descent" methods because the overall step in w_t is proportional to the negative gradient of the example's squared error (9.4). This is the direction in which the error falls most rapidly. Gradient descent methods are called "stochastic" when the update is done, as here, on only a single example, which might have been selected stochastically. Over many examples, making small steps, the overall effect is to minimize an average performance measure such as the $\overline{VE}$.

We turn now to the case in which the target output, here denoted $U_t \in \mathbb{R}$, of the tth training example, $S_t \to U_t$, is not the true value, $v_\pi(S_t)$, but some, possibly random, approximation to it. For example, U_t might be a noise-corrupted version of $v_\pi(S_t)$, or it might be one of the bootstrapping targets using $\hat{v}$ mentioned in the previous section. In these cases we cannot perform the exact update (9.5) because $v_\pi(S_t)$ is unknown, but we can approximate it by substituting U_t in place of $v_\pi(S_t)$. This yields the following general SGD method for state-value prediction:

$$w_{t+1} \doteq w_t + \alpha [U_t - \hat{v}(S_t, w_t)] \nabla \hat{v}(S_t, w_t)$$

If U_t is an unbiased estimate, that is, if $\mathbb{E}[U_t | S_t = s] = v_\pi(s)$, for each t, then w_t is guaranteed to converge to a local optimum under the usual stochastic approximation conditions (2.7) for decreasing α.

One does not obtain the same guarantees if a bootstrapping estimate of $v_\pi(S_t)$ is used as the target U_t in (9.7). Bootstrapping targets such as n-step returns $G_{t:t+n}$ or the DP target $\sum_{a,s',r} \pi(a|S_t) p(s', r | S_t, a)[r + \hat{v}(s', w_t)]$ all depend on the current value of the weight vector w_t, which implies that they will be biased and that they will not produce a true gradient-descent method. One way to look at this is that the key step from (9.4) to (9.5) relies on

the target being independent of w_t. This step would not be valid if a bootstrapping estimate were used in place of $v_\pi(S_t)$. Bootstrapping methods are not in fact instances of true gradient descent (Barnard, 1993). They take into account the effect of changing the weight vector w_t on the estimate, but ignore its effect on the target. They include only a part of the gradient and, accordingly, we call them semi-gradient methods.

Although semi-gradient (bootstrapping) methods do not converge as robustly as gradient methods, they do converge reliably in important cases such as the linear case discussed in the next section. Moreover, they offer important advantages that make them often clearly preferred. One reason for this is that they typically enable significantly faster learning, as we have seen in Chapters 6 and 7. Another is that they enable learning to be continual and online, without waiting for the end of an episode. This enables them to be used on continuing problems and provides computational advantages. A prototypical semi-gradient method is semi-gradient TD(0), which uses $U_t \doteq R_{t+1} + \hat{v}(S_{t+1}, w)$ as its target.

8.3.1 Stochastic Approximation and Mean-Square Convergence

Many reinforcement learning algorithms can be interpreted as instances of **stochastic approximation** (SA), that is, iterative methods for finding the root of an unknown function when only noisy observations are available. In its classical Robbins–Monro form, stochastic approximation considers the problem of solving

$$f(x) = 0,$$

when the learner does not observe $f(x_n)$ exactly, but only a noisy quantity

$$Y_n = f(x_n) + M_{n+1},$$

where M_{n+1} is a random noise term with zero conditional mean. The generic update is then

$$x_{n+1} = x_n - \alpha_n Y_n,$$

where $\alpha_n > 0$ is the step size.

A particularly important case is the **affine** one, in which

$$f(x) = Lx - b,$$

with $L \in \mathbb{R}^{d\times d}$ and $b \in \mathbb{R}^d$. If L is invertible, then the equation $f(x) = 0$ has the unique solution

$$x^* = L^{-1}b.$$

In this case, the noisy observation can be written as

$$Y_n = Lx_n - b + M_{n+1} = L(x_n - x^*) + M_{n+1},$$

and the stochastic approximation update becomes

$$x_{n+1} = x_n - \alpha_n\big(L(x_n - x^*) + M_{n+1}\big).$$

To study convergence, it is convenient to introduce the error

$$e_n = x_n - x^*.$$

Subtracting x^* from both sides of the update yields the **error recursion**

$$e_{n+1} = (I - \alpha_n L)e_n - \alpha_n M_{n+1}.$$

This expression makes clear the two forces acting on the iterate:

- the deterministic term $(I - \alpha_n L)e_n$, which contracts the error toward zero;
- the stochastic term $-\alpha_n M_{n+1}$, which perturbs the dynamics.

We now state standard assumptions ensuring convergence in mean square:

1. **Positive definiteness:** the matrix L is strictly positive definite, with minimum eigenvalue $\lambda_{\min} > 0$;
2. **Zero-mean noise:**
$$\mathbb{E}[M_{n+1} \mid \mathcal{F}_n] = 0;$$
3. **Bounded conditional second moment:**
$$\mathbb{E}[\|M_{n+1}\|^2 \mid \mathcal{F}_n] \le \sigma^2;$$
4. **Robbins–Monro step sizes:**
$$\sum_{n=0}^{\infty} \alpha_n = \infty, \qquad \sum_{n=0}^{\infty} \alpha_n^2 < \infty.$$

Under these assumptions, the stochastic approximation iterates converge to the true root in mean square.

Theorem. Under the assumptions above,

$$\lim_{n\to\infty} \mathbb{E}\big[\|x_n - x^*\|^2\big] = 0.$$

Proof sketch. Starting from

$$e_{n+1} = (I - \alpha_n L)e_n - \alpha_n M_{n+1},$$

we square both sides and take the conditional expectation with respect to $\mathcal{F}_n$. Since e_n is $\mathcal{F}_n$-measurable and the noise has zero conditional mean, the cross term vanishes:

$$\mathbb{E}\big[\|e_{n+1}\|^2 \mid \mathcal{F}_n\big] = \|(I - \alpha_n L)e_n\|^2 + \alpha_n^2 \mathbb{E}\big[\|M_{n+1}\|^2 \mid \mathcal{F}_n\big].$$

Using the positive definiteness of L, for sufficiently large n we obtain the bound

$$\|(I - \alpha_n L)e_n\|^2 \le (1 - \alpha_n \lambda_{\min})\|e_n\|^2,$$

hence

$$\mathbb{E}\big[\|e_{n+1}\|^2 \mid \mathcal{F}_n\big] \le (1 - \alpha_n \lambda_{\min})\|e_n\|^2 + \sigma^2 \alpha_n^2.$$

Taking total expectation and defining

$$v_n = \mathbb{E}[\|e_n\|^2],$$

we get the scalar recurrence

$$v_{n+1} \le (1 - \alpha_n \lambda_{\min}) v_n + \sigma^2 \alpha_n^2.$$

Rearranging,

$$v_{n+1} - v_n \le -\lambda_{\min} \alpha_n v_n + \sigma^2 \alpha_n^2.$$

Summing over n gives

$$\lambda_{\min} \sum_{n=N_0}^{\infty} \alpha_n v_n < \infty.$$

Because $\sum_n \alpha_n = \infty$ while $\sum_n \alpha_n^2 < \infty$, the sequence v_n must be driven arbitrarily close to zero, and the total possible upward fluctuation becomes negligible. Therefore,

$$\lim_{n\to\infty} v_n = 0,$$

that is,

$$\lim_{n\to\infty} \mathbb{E}[\|x_n - x^*\|^2] = 0.$$

This result is important in reinforcement learning because many update rules, especially in temporal-difference learning and approximate methods, have the general form of a stochastic approximation recursion. Stochastic approximation therefore provides the mathematical framework for analyzing the convergence of noisy iterative learning updates.

8.4 Linear Methods

One of the most important special cases of function approximation is that in which the approximate function, $\hat{v}(\cdot, w)$, is a linear function of the weight vector, w. Corresponding to every state s, there is a real-valued vector $x(s) \doteq (x_1(s), x_2(s), \ldots, x_d(s))^T$, with the same number of components as w. Linear methods approximate the state-value function by the inner product between w and $x(s)$:

$$\hat{v}(s, w) \doteq w^T x(s) \doteq \sum_{i=1}^{d} w_i x_i(s)$$

In this case the approximate value function is said to be linear in the weights, or simply linear.

The vector $x(s)$ is called a feature vector representing state s. Each component $x_i(s)$ of $x(s)$ is the value of a function $x_i : S \to \mathbb{R}$. We think of a feature as the entirety of one of these functions, and we call its value for a state s a feature of s. For linear methods, features are basis functions because they form a linear basis for the set of approximate functions. Constructing d-dimensional feature vectors to represent states is the same as selecting a set of d basis functions. Features may be defined in many different ways; we cover a few possibilities in the next sections.

It is natural to use SGD updates with linear function approximation. The gradient of the approximate value function with respect to w in this case is

$$\nabla \hat{v}(s, w) = x(s)$$

Thus, in the linear case the general SGD update (9.7) reduces to a particularly simple form:

$$w_{t+1} \doteq w_t + \alpha[U_t - \hat{v}(S_t, w_t)]x(S_t)$$

Because it is so simple, the linear SGD case is one of the most favorable for mathematical analysis. Almost all useful convergence results for learning systems of all kinds are for linear (or simpler) function approximation methods.

In particular, in the linear case there is only one optimum (or, in degenerate cases, one set of equally good optima), and thus any method that is guaranteed to converge to or near a local optimum is automatically guaranteed to converge to or near the global optimum. For example, the gradient Monte Carlo algorithm presented in the previous section converges to the global optimum of the $\overline{VE}$ under linear function approximation if α is reduced over time according to the usual conditions.

The semi-gradient TD(0) algorithm presented in the previous section also converges under linear function approximation, but this does not follow from general results on SGD; a separate theorem is necessary. The weight vector converged to is also not the global optimum, but rather a point near the local optimum. It is useful to consider this important case in more detail, specifically for the continuing case. The update at each time t is

$$w_{t+1} \doteq w_t + \alpha(R_{t+1} + w_t^T x_{t+1} - w_t^T x_t)x_t$$

$$= w_t + \alpha(R_{t+1}x_t - x_t(x_t - x_{t+1})^T w_t)$$

where here we have used the notational shorthand $x_t = x(S_t)$. Once the system has reached steady state, for any given w_t, the expected next weight vector can be written

$$\mathbb{E}[w_{t+1}|w_t] = w_t + \alpha(b - Aw_t)$$

where

$$b \doteq \mathbb{E}[R_{t+1}x_t] \in \mathbb{R}^d \quad \text{and} \quad A \doteq \mathbb{E}[x_t(x_t - \gamma x_{t+1})^T] \in \mathbb{R}^{d \times d}$$

From (9.10) it is clear that, if the system converges, it must converge to the weight vector w_{TD} at which

$$b - Aw_{TD} = 0$$
$$\Rightarrow b = Aw_{TD}$$
$$\Rightarrow w_{TD} \doteq A^{-1}b$$

This quantity is called the TD fixed point. In fact linear semi-gradient TD(0) converges to this point. Some of the theory proving its convergence, and the existence of the inverse above, is given in the box.

At the TD fixed point, it has also been proven (in the continuing case) that the $\overline{VE}$ is within a bounded expansion of the lowest possible error:

$$\overline{VE}(w_{TD}) \leq \frac{1}{1-\gamma} \min_w \overline{VE}(w)$$

That is, the asymptotic error of the TD method is no more than $\frac{1}{1-\gamma}$ times the smallest possible error, that attained in the limit by the Monte Carlo method. Because γ is often near one, this expansion factor can be quite large, so there is substantial potential loss in asymptotic performance with the TD method. On the other hand, recall that the TD methods are often of vastly reduced variance compared to Monte Carlo methods, and thus faster, as we saw in Chapters 6 and 7. Which method will be best depends on the nature of the approximation and problem, and on how long learning continues.

8.5 Feature Construction for Linear Methods

Linear methods are interesting because of their convergence guarantees, but also because in practice they can be very efficient in terms of both data and computation. Whether or not this is so depends critically on how the states are represented in terms of features, which we investigate in this large section. Choosing features appropriate to the task is an important way of adding prior domain knowledge to reinforcement learning systems.

Intuitively, the features should correspond to the aspects of the state space along which generalization may be appropriate. If we are valuing geometric objects, for example, we might want to have features for each possible shape, color, size, or function. If we are valuing states of a mobile robot, then we might want to have features for locations, degrees of remaining battery power, recent sonar readings, and so on.

A limitation of the linear form is that it cannot take into account any interactions between features, such as the presence of feature i being good only in the absence of feature j. For example, in the pole-balancing task (Example 3.4), high angular velocity can be either good or bad depending on the angle. If the angle is high, then high angular velocity means an imminent danger of falling—a bad state—whereas if the angle is low, then high angular velocity means the pole is righting itself—a good state. A linear value function could not represent this if its features coded separately for the angle and the angular velocity. It needs instead, or in addition, features for combinations of these two underlying state dimensions. In the following subsections we consider a variety of general ways of doing this.

- **Polynomials.** You can represent a state with n dimensions, so that $x(s) = (s_1, s_2, \ldots, s_n)^T$, maybe with interactions or appending a constant value (1) at the beginning to represent affine functions in the original state numbers. Thus, each order-n polynomial-basis feature x_i can be written as

$$x_i(s) = \prod_{j=1}^{n} s_j^{c_{i,j}} \tag{8.1}$$

  where $c_{i,j}$ is an integer in the set $\{0, 1, \ldots, n\}$ for an integer $n \geq 0$.
- **Fourier Basis.** The Fourier basis represents functions as a sum of sinusoids. For a state s, we can define Fourier features as

$$x_i(s) = \cos(\omega_i^T s + \phi_i) \tag{8.2}$$

  where ω_i is a frequency vector and ϕ_i is a phase shift. This allows for capturing periodic patterns in the state space.
- **Radial Basis Functions (RBFs).** RBFs are a popular choice for function approximation, defined as

$$x_i(s) = \exp\left(-\frac{1}{2\sigma^2}\|s - \mu_i\|^2\right) \tag{8.3}$$

  where μ_i is the center of the RBF and σ controls the width. RBFs are particularly useful for capturing localized features in the state space.
- **ANN.** Artificial Neural Networks (ANNs) can be used to learn complex feature representations from raw state inputs. They consist of multiple layers of interconnected nodes, where each node applies a non-linear transformation to its inputs. The learned features can capture intricate patterns and interactions in the state space.

Chapter 9

Policy Gradient Methods

So far in this book almost all the methods have been *action-value methods*; they learned the values of actions and then selected actions based on their estimated action values. In this chapter we consider methods that instead learn a *parameterized policy* that can select actions without consulting a value function. A value function may still be used to *learn* the policy parameter, but it is not required for action selection. We will use the notation $\theta \in \mathbb{R}^{d'}$ for the policy's parameter vector.

In this chapter we consider methods for learning the policy parameter based on the gradient of some scalar performance measure $J(\theta)$ with respect to the policy parameter. These methods seek to *maximize* the performance, so their updates approximate gradient *ascent* in J:

$$\theta_{t+1} = \theta_t + \alpha \nabla \hat{J}(\theta_t),$$

All methods that follow this general schema are called *policy gradient methods*.

9.1 Policy Approximation and its Advantages

In policy gradient methods, the policy can be parameterized in any way, as long as $\pi(a|s,\theta)$ is differentiable with respect to its parameters, i.e., as long as $\nabla\pi(a|s,\theta)$ exists and is finite for all $s \in \mathcal{S}, a \in \mathcal{A}(s), \theta \in \mathbb{R}^{d'}$.

If the action space is discrete and not too large, then a natural and common kind of parameterization is to form parameterized numerical preferences $h(s,a,\theta) \in \mathbb{R}$ for each state-action pair. The actions with the highest preferences in each state are given the highest probabilities of being selected, for example, according to an exponential soft-max distribution:

$$\pi(a|s,\theta) = \frac{e^{h(s,a,\theta)}}{\sum_{a'} e^{h(s,a',\theta)}}$$

The action preferences themselves can be parameterized arbitrarily. As an example, they might be computed by a deep artificial neural network (ANN), where θ is the vector of all the connection weights of the network. Or the preferences could be simply linear in features:

$$h(s,a,\theta) = \theta^\top \phi(s,a)$$

9.2 Policy Gradient Theorem

We can simplify the notation without losing any meaningful generality by assuming that every episode starts in some particular (non-random) state s_0. Then, in the episodic case we define performance as:

$$J(\theta) = v_{\pi_\theta}(s_0)$$

where v_{π_θ} is the true value function for π_θ, the policy determined by θ. From here on in our discussion we will assume no discounting ($\gamma = 1$) for the episodic case.

With function approximation it may seem challenging to change the policy parameter in a way that ensures improvement. The problem is that performance depends on both the action selections and the distribution of states in which those selections are made, and that both of these are affected by the policy parameter. Given a state, the effect of the policy parameter on the actions, and thus on the reward, can be computed in a relatively

straightforward way from knowledge of the parameterization. But the effect of the policy on the state distribution is a function of the environment and is typically unknown. How can we estimate the performance gradient with respect to the policy parameter when the gradient depends on the unknown effect of policy changes on the state distribution?

Here comes the *policy gradient theorem*, which provides an analytical expression for the gradient of performance with respect to the policy parameter. The policy gradient theorem for the episodic case establishes that

$$\nabla J(\theta) \propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\theta),$$

The distribution μ here (seen in Chapters 8 and 9) is the on-policy distribution under π.

9.3 REINFORCE: Monte Carlo Policy Gradient

Recall our strategy for stochastic gradient ascent, which requires a way to obtain samples such that the expectation of the sample gradient is proportional to the actual gradient of the performance measure as a function of the parameter. The sample gradient need only be proportional to the gradient because any constant of proportionality can be absorbed into the step size α, which is otherwise arbitrary.

Notice that the right-hand side of the policy gradient theorem is a sum over states weighted by how often the states occur under the target policy π; if π is followed, then states will be encountered in these proportions. Thus:

$$\begin{aligned}
\nabla J(\theta) &\propto \sum_s \mu(s) \sum_a q_\pi(s,a) \nabla \pi(a|s,\theta) \\
&= \mathbb{E}_\pi \left[\sum_a q_\pi(S_t,a) \nabla \pi(a|S_t,\theta) \right]
\end{aligned}$$

We could stop here and instantiate our stochastic gradient-ascent algorithm as

$$\theta_{t+1} = \theta_t + \alpha \sum_a \hat{q}(S_t,a,w) \nabla \pi(a|S_t,\theta)$$

but our current interest is the classical REINFORCE algorithm whose update at time t involves just A_t, the one action actually taken at time t.

We continue our derivation of REINFORCE by introducing A_t in the same way as we introduced S_t, i.e., by replacing a sum over the random variable's possible values by an expectation under π, and then sampling the expectation.

$$\begin{aligned}
\nabla J(\theta) &\propto \mathbb{E}_\pi \left[\sum_a \pi(a|S_t,\theta) q_\pi(S_t,a) \frac{\nabla \pi(a|S_t,\theta)}{\pi(a|S_t,\theta)} \right] \\
&= \mathbb{E}_\pi \left[q_\pi(S_t,A_t) \frac{\nabla \pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)} \right] \\
&= \mathbb{E}_\pi \left[G_t \frac{\nabla \pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)} \right] \qquad \text{(because } \mathbb{E}_\pi \left[G_t | S_t, A_t \right] = q_\pi(S_t,A_t)\text{)}
\end{aligned}$$

where G_t is the return as usual. The final expression in brackets is exactly what is needed, a quantity that can be sampled on each time step whose expectation is proportional to the gradient. Using this sample to instantiate our generic stochastic gradient-ascent algorithm yields the REINFORCE update:

$$\theta_{t+1} = \theta_t + \alpha G_t \frac{\nabla \pi(A_t|S_t,\theta)}{\pi(A_t|S_t,\theta)}$$

The vector is the direction in parameter space that most increases the probability of repeating the action A_t on future visits to state S_t. The update increases the parameter vector in this direction proportional to the return, and inversely proportional to the action probability. The former makes sense because it causes the parameter to move most in the directions that favour actions that yield the highest return. The latter makes sense because otherwise actions that are selected frequently are at an advantage and might win out even if they do not yield the highest return.

REINFORCE: Monte-Carlo Policy-Gradient Control (episodic) for π_*

Input: a differentiable policy parameterization $\pi(a|s, \boldsymbol{\theta})$
Algorithm parameter: step size $\alpha > 0$
Initialize policy parameter $\boldsymbol{\theta} \in \mathbb{R}^{d'}$ (e.g., to $\mathbf{0}$)

Loop forever (for each episode):
  Generate an episode $S_0, A_0, R_1, \ldots, S_{T-1}, A_{T-1}, R_T$, following $\pi(\cdot|\cdot, \boldsymbol{\theta})$
  Loop for each step of the episode $t = 0, 1, \ldots, T-1$:
    $G \leftarrow \sum_{k=t+1}^{T} \gamma^{k-t-1} R_k$ $\quad (G_t)$
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \gamma^t G \nabla \ln \pi(A_t|S_t, \boldsymbol{\theta})$

Figure 9.1: Monte-Carlo Policy-Gradient Control (episodic) for π_*

As a stochastic gradient method, REINFORCE has good theoretical convergence properties. By construction, the expected update over an episode is in the same direction as the performance gradient. This assures an improvement in expected performance for sufficiently small α, and convergence to a local optimum under standard stochastic approximation conditions for decreasing α. However, as Monte Carlo method REINFORCE may be of high variance and thus produce slow learning.

9.3.1 REINFORCE with Baseline

The REINFORCE update derived above is unbiased, but it can suffer from high variance because the return G_t depends on all future rewards in the trajectory. A standard way to reduce this variance is to subtract a baseline from the return.

The update becomes:

$$\theta_{t+1} = \theta_t + \alpha\big(G_t - b(S_t)\big)\frac{\nabla \pi(A_t \mid S_t, \theta)}{\pi(A_t \mid S_t, \theta)}.$$

Equivalently, using the log-derivative identity,

$$\nabla_\theta \log \pi_\theta(a \mid s) = \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)},$$

we can write:

$$\theta_{t+1} = \theta_t + \alpha\big(G_t - b(S_t)\big)\nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

The baseline $b(S_t)$ can be any function of the state, as long as it does not depend on the specific action A_t selected at time t. The reason is that subtracting such a baseline does not change the expected policy gradient:

$$\mathbb{E}_{a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s) b(s)\right] = \sum_{a\in\mathcal{A}} \pi_\theta(a \mid s)\frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} b(s).$$

Therefore,

$$\mathbb{E}_{a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s) b(s)\right] = b(s)\sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a \mid s).$$

Since the probabilities of all actions must sum to one,

$$\sum_{a\in\mathcal{A}} \pi_\theta(a \mid s) = 1,$$

we have

$$\sum_{a\in\mathcal{A}} \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta \sum_{a\in\mathcal{A}} \pi_\theta(a \mid s) = \nabla_\theta 1 = 0.$$

Thus,

$$\mathbb{E}_{a\sim\pi_\theta}\left[\nabla_\theta \log \pi_\theta(a \mid s) b(s)\right] = 0.$$

This shows that the baseline does not introduce bias into the gradient estimate. It only changes the variance. A common and useful choice is to use the state-value function as the baseline:

$$b(S_t) = v_\pi(S_t).$$

In practice, $v_\pi(S_t)$ is usually unknown and is approximated by a parameterized value function $\hat{v}(S_t, w)$.

9.3.2 Advantage Function

Using the state-value function as a baseline leads naturally to the advantage function. The advantage of taking action a in state s under policy π is defined as:

$$A^\pi(s,a) = q_\pi(s,a) - v_\pi(s).$$

The value $q_\pi(s,a)$ measures the expected return after taking action a in state s and then following policy π. The value $v_\pi(s)$ measures the expected return from state s when following the policy normally.

Therefore, the advantage function measures how much better or worse action a is compared to the average action selected by the policy in state s.

Using the advantage function, the policy gradient can be written as:

$$\nabla J(\theta) \propto \mathbb{E}_\pi \left[A^\pi(S_t, A_t) \nabla_\theta \log \pi_\theta(A_t \mid S_t) \right].$$

This form is useful because it updates the policy according to whether the selected action was better or worse than expected:

- if $A^\pi(S_t, A_t) > 0$, the probability of action A_t is increased;
- if $A^\pi(S_t, A_t) < 0$, the probability of action A_t is decreased;
- if $A^\pi(S_t, A_t) = 0$, the action performed as expected and the update is small or zero.

9.3.3 Actor-Critic Methods

REINFORCE is a Monte Carlo method. This means that it must wait until the end of an episode to compute the return G_t. Although this gives an unbiased estimate, it can produce high variance and slow learning.

Actor-critic methods combine policy-gradient methods with temporal-difference learning. The method has two components:

- the **actor**, which is the parameterized policy $\pi_\theta(a \mid s)$;
- the **critic**, which estimates the value function, usually $\hat{v}(s, w)$.

The actor decides which action to take, while the critic evaluates how good the action was. Instead of using the full Monte Carlo return G_t, the critic uses a one-step TD error:

$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w).$$

This TD error can be interpreted as an estimate of the advantage function:

$$\delta_t \approx A^\pi(S_t, A_t).$$

Indeed, if the value-function approximation is accurate, then:

$$\mathbb{E}[\delta_t \mid S_t, A_t] \approx q_\pi(S_t, A_t) - v_\pi(S_t) = A^\pi(S_t, A_t).$$

The critic is updated using a semi-gradient TD update:

$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w \hat{v}(S_t, w).$$

The actor is updated in the policy-gradient direction, using the TD error as the advantage estimate:

$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

Therefore, the actor-critic algorithm proceeds as follows:

1. observe the current state S_t;
2. sample an action $A_t \sim \pi_\theta(\cdot \mid S_t)$;
3. observe the reward R_{t+1} and the next state S_{t+1};
4. compute the TD error
$$\delta_t = R_{t+1} + \gamma \hat{v}(S_{t+1}, w) - \hat{v}(S_t, w);$$
5. update the critic:
$$w_{t+1} = w_t + \alpha_w \delta_t \nabla_w \hat{v}(S_t, w);$$
6. update the actor:
$$\theta_{t+1} = \theta_t + \alpha_\theta \delta_t \nabla_\theta \log \pi_\theta(A_t \mid S_t).$$

The advantage of actor-critic methods is that they can learn online, step by step, without waiting for the end of the episode. Compared with REINFORCE, they usually reduce variance because they replace the full return G_t with a bootstrapped estimate. However, because the critic is only an approximation of the true value function, actor-critic methods can introduce bias.

Chapter 10

Advanced Policy Gradient Notes

10.1 Derivation of the Policy Gradient Theorem

10.1.1 Setup and Parameterization

We consider the same Markov Decision Process (MDP) notation as before. However, instead of optimizing over the space of all possible policies, we assume the policy is parameterized by a continuous vector $\theta \in \mathbb{R}^d$.

Let $\pi_\theta(a \mid s)$ denote the probability of taking action a in state s under parameters θ. The goal is to maximize the expected discounted return:

$$J(\theta) = r^T (I - \gamma P^{\pi_\theta})^{-1} \rho \tag{10.1}$$

where:

- P^{π_θ} is the $|S| \times |S|$ left-stochastic transition matrix with entries

$$P^{\pi_\theta}_{s',s} = \sum_a p(s' \mid s, a) \pi_\theta(a \mid s).$$

- r is the $|S| \times 1$ column vector of state rewards.
- ρ is the $|S| \times 1$ column vector of the initial state distribution.
- $\gamma \in [0, 1)$ is the discount factor.

As established previously, we define the state-value row vector

$$V^{\pi_\theta} = r^T (I - \gamma P^{\pi_\theta})^{-1}$$

and the discounted state visitation column vector

$$\mu^{\pi_\theta} = (I - \gamma P^{\pi_\theta})^{-1} \rho.$$

Definition 1 (State-Action Value Function). The expected return of taking action a in state s, and thereafter following π_θ, is defined as:

$$Q^{\pi_\theta}(s, a) = r_s + \gamma \sum_{s' \in S} p(s' \mid s, a) V^{\pi_\theta}_{s'}. \tag{10.2}$$

10.1.2 Matrix Calculus Identity

To compute the gradient $\nabla_\theta J(\theta)$, we need to differentiate the inverse of a matrix that depends on θ. We rely on a fundamental identity from matrix calculus.

Lemma 1 (Derivative of an Inverse Matrix). For an invertible matrix $A(\theta)$ parameterized by θ, the gradient of its inverse is:

$$\nabla_\theta A^{-1}(\theta) = -A^{-1}(\theta) \left(\nabla_\theta A(\theta)\right) A^{-1}(\theta). \tag{10.3}$$

In our context, $A(\theta) = I - \gamma P^{\pi_\theta}$. Because the identity matrix I does not depend on θ, we have $\nabla_\theta A(\theta) = -\gamma \nabla_\theta P^{\pi_\theta}$. Substituting this into the lemma yields:

$$\nabla_\theta (I - \gamma P^{\pi_\theta})^{-1} = \gamma (I - \gamma P^{\pi_\theta})^{-1} (\nabla_\theta P^{\pi_\theta}) (I - \gamma P^{\pi_\theta})^{-1}. \tag{10.4}$$

10.1.3 Derivation of the Policy Gradient Theorem

Theorem 1 (Policy Gradient Theorem). For any differentiable policy π_θ, the gradient of the objective $J(\theta)$ is:

$$\nabla_\theta J(\theta) = \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a). \tag{10.5}$$

Proof. We begin by applying the gradient operator to the objective function:

$$\nabla_\theta J(\theta) = \nabla_\theta \left[r^T (I - \gamma P^{\pi_\theta})^{-1} \rho \right] = r^T \left[\nabla_\theta (I - \gamma P^{\pi_\theta})^{-1} \right] \rho.$$

Substituting the matrix identity from Equation 4:

$$\begin{aligned} \nabla_\theta J(\theta) &= r^T \left[\gamma (I - \gamma P^{\pi_\theta})^{-1} (\nabla_\theta P^{\pi_\theta}) (I - \gamma P^{\pi_\theta})^{-1} \right] \rho \\ &= \gamma \left[r^T (I - \gamma P^{\pi_\theta})^{-1} \right] (\nabla_\theta P^{\pi_\theta}) \left[(I - \gamma P^{\pi_\theta})^{-1} \rho \right]. \end{aligned}$$

Recognizing our definitions for V^{π_θ} and μ^{π_θ}, this dramatically simplifies to a vector-matrix-vector product:

$$\nabla_\theta J(\theta) = \gamma V^{\pi_\theta} (\nabla_\theta P^{\pi_\theta}) \mu^{\pi_\theta}. \tag{10.6}$$

Now, we expand Equation 6 into its scalar sums. The element at row s' and column s of the matrix $\nabla_\theta P^{\pi_\theta}$ is given by $\sum_a p(s' \mid s, a) \nabla_\theta \pi_\theta(a \mid s)$.

$$\begin{aligned} \nabla_\theta J(\theta) &= \gamma \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{s' \in S} V_{s'}^{\pi_\theta} \sum_{a \in A} p(s' \mid s, a) \nabla_\theta \pi_\theta(a \mid s) \\ &= \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) \left[\gamma \sum_{s' \in S} p(s' \mid s, a) V_{s'}^{\pi_\theta} \right]. \end{aligned}$$

Notice that the term in the brackets is exactly $Q^{\pi_\theta}(s, a) - r_s$. Substituting this back into the sum:

$$\begin{aligned} \nabla_\theta J(\theta) &= \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) \left(Q^{\pi_\theta}(s, a) - r_s \right) \\ &= \sum_{s \in S} \mu_s^{\pi_\theta} \left(\sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) - r_s \sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) \right). \end{aligned}$$

Finally, recall that probabilities must sum to 1, meaning $\sum_a \pi_\theta(a \mid s) = 1$. The derivative of a constant is zero, so $\sum_a \nabla_\theta \pi_\theta(a \mid s) = \nabla_\theta(1) = 0$. Consequently, the term involving the state reward r_s completely vanishes:

$$\nabla_\theta J(\theta) = \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{a \in A} \nabla_\theta \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a). \tag{10.7}$$

10.1.4 The Log-Derivative Trick

To make this theorem practical for estimation via sampling, we utilize the log-derivative trick:

$$\nabla_\theta \pi_\theta(a \mid s) = \pi_\theta(a \mid s) \frac{\nabla_\theta \pi_\theta(a \mid s)}{\pi_\theta(a \mid s)} = \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s). \tag{10.8}$$

Substituting this into the Policy Gradient Theorem yields an expectation over trajectories generated by the policy:

$$\nabla_\theta J(\theta) = \sum_{s \in S} \mu_s^{\pi_\theta} \sum_{a \in A} \pi_\theta(a \mid s) \nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) \tag{10.9}$$

which is commonly written as

$$\mathbb{E}_{s \sim \mu^{\pi_\theta},\, a \sim \pi_\theta} \left[\nabla_\theta \log \pi_\theta(a \mid s) Q^{\pi_\theta}(s, a) \right].$$

10.2 Global Optima in Markov Decision Processes

10.2.1 Background and Notation

We consider a Markov Decision Process (MDP) defined by a finite state space S, a finite action space A, a transition dynamics model $p(s' \mid s, a)$, and a general reward function $r(s, a)$, which represents the expected immediate reward received when taking action a in state s.

A stochastic policy $\pi(a \mid s)$ defines the probability of taking action a in state s. Given a policy π, we can compute the expected immediate reward for each state.

Definition 1 (Expected Reward Vector). Let $r^\pi \in \mathbb{R}^{|S|\times 1}$ be the column vector of expected immediate rewards under policy π. The entry for state s is:

$$r_s^\pi = \sum_{a\in A} \pi(a \mid s) r(s,a). \tag{10.10}$$

Definition 2 (Left-Stochastic Transition Matrix). Let $P^\pi \in \mathbb{R}^{|S|\times|S|}$ be the transition probability matrix induced by π. Using the column-to-row (left-stochastic) convention, the entry in row s' and column s is:

$$P^\pi_{s',s} = \sum_{a\in A} p(s' \mid s,a)\pi(a \mid s). \tag{10.11}$$

Let $\rho \in \mathbb{R}^{|S|\times 1}$ be the initial state probability distribution vector. The objective function is the expected sum of discounted rewards, written in matrix-vector form:

$$J(\pi) = (r^\pi)^T (I - \gamma P^\pi)^{-1}\rho \tag{10.12}$$

where $\gamma \in [0,1)$ is the discount factor.

Definition 3 (State Value Row-Vector). We define $V^\pi \in \mathbb{R}^{1\times|S|}$ as the row vector of expected discounted returns:

$$V^\pi = (r^\pi)^T (I - \gamma P^\pi)^{-1} \qquad \Longrightarrow \qquad V^\pi = (r^\pi)^T + \gamma V^\pi P^\pi. \tag{10.13}$$

Definition 4 (Discounted Visitation Column-Vector). We define $\mu^\pi \in \mathbb{R}^{|S|\times 1}$ as the column vector of expected discounted state visitations:

$$\mu^\pi = (I - \gamma P^\pi)^{-1}\rho \qquad \Longrightarrow \qquad \mu^\pi = \rho + \gamma P^\pi \mu^\pi. \tag{10.14}$$

Definition 5 (State-Action Value Function). The expected return of taking action a in state s, and thereafter following π, is:

$$Q^\pi(s,a) = r(s,a) + \gamma \sum_{s'\in S} p(s' \mid s,a) V^\pi_{s'}. \tag{10.15}$$

10.2.2 The Performance Difference Lemma

Lemma 1 (Performance Difference). For any two policies π and π', the exact difference in their expected returns is:

$$J(\pi') - J(\pi) = \sum_{s\in S} \mu_s^{\pi'} \sum_{a\in A} \left(\pi'(a \mid s) - \pi(a \mid s)\right) Q^\pi(s,a). \tag{10.16}$$

Proof. Using the identity $V^{\pi'}(I - \gamma P^{\pi'}) = (r^{\pi'})^T$, we evaluate the difference between $V^{\pi'}$ and V^π:

$$V^{\pi'} - \gamma V^{\pi'} P^{\pi'} = (r^{\pi'})^T$$

$$V^{\pi'} - V^\pi = (r^{\pi'})^T - (r^\pi)^T + \gamma V^{\pi'} P^{\pi'} - \gamma V^\pi P^\pi.$$

We add and subtract $\gamma V^\pi P^{\pi'}$ on the right side to factor out the value functions:

$$V^{\pi'} - V^\pi = (r^{\pi'})^T - (r^\pi)^T + \gamma(V^{\pi'} - V^\pi)P^{\pi'} + \gamma V^\pi(P^{\pi'} - P^\pi)$$

$$(V^{\pi'} - V^\pi)(I - \gamma P^{\pi'}) = (r^{\pi'})^T - (r^\pi)^T + \gamma V^\pi(P^{\pi'} - P^\pi)$$

$$V^{\pi'} - V^\pi = \left[(r^{\pi'})^T - (r^\pi)^T + \gamma V^\pi(P^{\pi'} - P^\pi)\right](I - \gamma P^{\pi'})^{-1}.$$

Post-multiplying by ρ yields the scalar objective difference:

$$J(\pi') - J(\pi) = \left[(r^{\pi'})^T - (r^\pi)^T + \gamma V^\pi(P^{\pi'} - P^\pi)\right]\mu^{\pi'}. \tag{10.17}$$

We now expand the term inside the brackets for a specific state s:

$$\begin{aligned}\text{Terms} &= \left(r_s^{\pi'} - r_s^\pi\right) + \gamma \sum_{s'\in S} V^\pi_{s'}\left(P^{\pi'}_{s',s} - P^\pi_{s',s}\right) \\ &= \sum_{a\in A}\left(\pi'(a \mid s) - \pi(a \mid s)\right) r(s,a) \\ &\quad + \gamma \sum_{s'\in S} V^\pi_{s'} \sum_{a\in A}\left(\pi'(a \mid s) - \pi(a \mid s)\right) p(s' \mid s,a).\end{aligned}$$

Factoring out the policy difference $(\pi'(a \mid s) - \pi(a \mid s))$:

$$\begin{aligned}\text{Terms} &= \sum_{a \in A} (\pi'(a \mid s) - \pi(a \mid s)) \left[r(s,a) + \gamma \sum_{s' \in S} p(s' \mid s,a) V_{s'}^{\pi} \right] \\ &= \sum_{a \in A} (\pi'(a \mid s) - \pi(a \mid s)) \, Q^{\pi}(s,a).\end{aligned}$$

Substituting this back into Equation 8 gives the final result:

$$J(\pi') - J(\pi) = \sum_{s \in S} \mu_s^{\pi'} \sum_{a \in A} (\pi'(a \mid s) - \pi(a \mid s)) \, Q^{\pi}(s,a). \tag{10.18}$$

10.2.3 Local and Global Optima

Theorem 1. If π is a local maximum of $J(\pi)$, then π is a global maximum.

Proof. Assume π is a local maximum. Consider a linear mixture towards any other policy π':

$$\pi_\alpha = \pi + \alpha(\pi' - \pi) \qquad \text{for } \alpha \in [0,1].$$

Applying the generalized Performance Difference Lemma and taking the limit as $\alpha \to 0^+$ (where $\mu^{\pi_\alpha} \to \mu^{\pi}$):

$$\lim_{\alpha \to 0^+} \frac{J(\pi_\alpha) - J(\pi)}{\alpha} = \sum_{s \in S} \mu_s^{\pi} \sum_{a \in A} (\pi'(a \mid s) - \pi(a \mid s)) \, Q^{\pi}(s,a) \leq 0. \tag{10.19}$$

Because the choice of $\pi'(\cdot \mid s)$ can be made completely independently for each state s, and assuming $\mu_s^{\pi} > 0$, the inner sum must be non-positive for every state:

$$\sum_{a \in A} \pi'(a \mid s) Q^{\pi}(s,a) \leq \sum_{a \in A} \pi(a \mid s) Q^{\pi}(s,a) \qquad \forall \pi', \forall s. \tag{10.20}$$

This indicates that the local maximum policy π satisfies the Bellman Optimality Equation: it places all probability mass on actions that maximize $Q^{\pi}(s,a)$.

Now, let π^* be an absolute global optimal policy. We apply the Performance Difference Lemma between π^* and our local maximum π:

$$J(\pi^*) - J(\pi) = \sum_{s \in S} \mu_s^{\pi^*} \left[\sum_{a \in A} \pi^*(a \mid s) Q^{\pi}(s,a) - \max_a Q^{\pi}(s,a) \right]. \tag{10.21}$$

Because the term in the brackets is mathematically forced to be ≤ 0 for every state, and $\mu_s^{\pi^*} \geq 0$, the sum is non-positive:

$$J(\pi^*) - J(\pi) \leq 0.$$

Since π^* is the global maximum, $J(\pi^*) \geq J(\pi)$. Thus,

$$J(\pi) = J(\pi^*).$$

Chapter 11

Multi-Agent Reinforcement Learning

11.1 General Introduction

Multi-agent reinforcement learning studies decision-making problems in which more than one agent interacts in the same environment. In this setting, the outcome of one agent's action depends not only on the state of the environment, but also on the actions selected by the other agents. As a result, the environment is no longer stationary from the perspective of a single learner: while one agent is improving its policy, the behavior of the others may change as well.

Multi-agent problems can have different structures:

- **cooperative**, when agents share the same objective and try to maximize a common return;
- **competitive**, when one agent's gain corresponds to another agent's loss, as in zero-sum games;
- **mixed**, when agents may have partially aligned and partially conflicting interests.

The main difficulty is that each agent must learn while adapting to the presence of other learners. This creates several challenges, including non-stationarity, coordination, credit assignment, and the possible need to model the behavior or intentions of other agents.

In cooperative settings, agents must discover policies that work well together, often without direct access to each other's internal state. In competitive settings, agents must instead learn robust strategies that perform well against strong opponents. For this reason, multi-agent reinforcement learning often draws from both reinforcement learning and game theory.

Typical applications include board games, robotic teams, autonomous driving, distributed control, network routing, and economic or strategic environments in which multiple decision-makers interact over time.

11.1.1 Formulation

In the multi-agent case, a policy is no longer associated with a single decision-maker, but with each agent in the system. If there are n agents, then the evolution of the environment depends on a *joint action*

$$(a^1, a^2, \ldots, a^n),$$

where a^i is the action selected by agent i. Similarly, rewards may be shared by all agents or may be different for each one, depending on whether the task is cooperative or competitive.

This means that the quality of one agent's policy cannot be evaluated independently from the policies of the others. A strategy that is good against one set of opponents may be poor against another, and in cooperative tasks an individually reasonable action may still be harmful if it is not coordinated with the team.

11.1.2 Common Approaches

One simple approach is to let each agent learn independently, treating the others as part of the environment. This is often easy to implement, but it ignores the fact that the environment is changing because the other agents are learning too.

A common alternative is **centralized training with decentralized execution**. During training, agents may use additional information such as the observations or actions of the other agents, which helps stabilize learning. At execution time, however, each agent must act using only the information available locally. This framework is widely used because it combines richer learning signals with realistic deployment constraints.

11.1.3 Relation with Game Theory

Multi-agent reinforcement learning is closely connected to game theory because agents are not only optimizing returns, but are also reacting to each other strategically. In competitive settings, an important goal is to learn policies that are hard to exploit. In cooperative settings, the focus is often on coordination and on reaching joint behaviors that are beneficial for the whole group.

For this reason, ideas such as equilibrium, best response, and opponent modeling often appear naturally in multi-agent RL. Reinforcement learning provides the mechanism for learning from experience, while game theory provides tools to reason about strategic interaction.

Exercise:

For each game, identify the main RL components of the problem and discuss which challenges arise from the presence of multiple agents.

	Chess	GO
State Space	Board configuration Castling rights Side to move En passant availability Draw rules Terminal state	Board configuration Side to move Ko-related information Terminal state
Actions	All legal moves in the current state Resign Offer a draw	All legal stone placements Pass Resign
Transition Probabilities	Deterministic, $\{0, 1\}$	Deterministic, $\{0, 1\}$
Rewards	Final outcome, or shaped intermediate rewards	Final outcome, or shaped intermediate rewards
Perfectly Observable	Yes	Yes
Techniques that could be used	Tree search Self-play Function approximation	Tree search Self-play Function approximation

	Pokemon
State Space	Turn number Remaining HP Status conditions Active Pokemon Available team members Remaining PP Items and field effects
Actions	Moves of the pokemon available with remaining energies Resign Use an Item Change Pokemon
Transition Probabilities	Not deterministic, probability of missing, some moves are stochastic
Rewards	Win/Lose, Damage Inflicted, How effective is the move, Damage Taken
Perfectly Observable	No, it is partially observable